OtterSec

# Helius Zolana ZK

Security Assessment

Prepared for

Helius

Prepared by

Otter Audits, LLC

Prepared on

November 4th, 2026


Himanshu Sheoran deuterium@osec.io

Andraž Strgar zeski@osec.io

Minsun Kim ahpuh@osec.io

# Table of Contents

1. Executive summary 3
1.1. Overview 3
1.2. Key findings 3
2. Scope 5
3. Findings 6
3.1. Findings summary 6
4. Advisories 7
OS-ZZK-ADV-00 Unbounded fake-GLV hints may bypass P256 ECDSA authorization 8
OS-ZZK-ADV-01 Unbounded prover HTTP intake permits resource exhaustion 12
OS-ZZK-ADV-02 Adaptive Bloom-filter coverage can selectively block a known nullifier 14
OS-ZZK-ADV-03 Deterministic private transaction hash exposes spent UTXO commitments 17
OS-ZZK-ADV-04 Incorrect threshold in 2-of-5 policy 21
OS-ZZK-ADV-05 Skipped protocol-config gate modes permit first-writer role capture 24
OS-ZZK-ADV-06 RingEddsa exact-intent authorization depends on trusted ring-program forwarding 27
OS-ZZK-ADV-07 Permissioned ring creation exposes the governance signer to candidate code 30
OS-ZZK-ADV-08 Cheap zero-value deposits can rotate the finite state-root history 33
OS-ZZK-ADV-09 Merge proof does not bind its output tree 36
OS-ZZK-ADV-10 The ring RPC cannot refresh an asset registry larger than 4,096 entries 40
OS-ZZK-ADV-11 Batch address-append circuit trusts inconsistent batch size and slice length 43
OS-ZZK-ADV-12 TypeScript tree layout constants omit the 64-byte reserved header 45
OS-ZZK-ADV-13 Cached tree-update replay policy conflicts 48
OS-ZZK-ADV-14 Address inputs bypass the dummy-input reserve gate 51
OS-ZZK-ADV-15 Mixed default- and policy-ring P256 inputs reveal the shared owner 54

© 2026 Otter Audits LLC. All Rights Reserved.

OS-ZZK-ADV-16 Duplicate UTXO commitments make Photon return ambiguous membership proofs 57

5. Suggestions 60

OS-ZZK-SUG-00 Impossible Poseidon injectivity axiom invalidates Lean hash-binding proofs 61

OS-ZZK-SUG-01 Merkle-root gadget trusts inconsistent height and slice lengths 63

OS-ZZK-SUG-02 Signed-amount range check accepts one unreachable boundary value 65

OS-ZZK-SUG-03 Merkle-tree loader returns mutable bytes after releasing the account borrow 67

OS-ZZK-SUG-04 Generic tree initialization can record capacities that do not match its memory layout 69

OS-ZZK-SUG-05 Ordinary non-sentinel existing-value lookup already returns the documented predecessor 72

OS-ZZK-SUG-06 A malicious prover cannot replace the client-bound dummy-output owner marker 75

OS-ZZK-SUG-07 Noncanonical output commitments panic instead of returning a program error 78

OS-ZZK-SUG-08 Non-inclusion verifier discards a false Merkle verification result 80

OS-ZZK-SUG-09 Proof helper accepts ciphertexts too large for the transaction wire format 83

OS-ZZK-SUG-10 Proof bundle omits the instruction discriminator and circuit identity 87

OS-ZZK-SUG-11 Proof helper emits decimal view tags as noncanonical hex strings 89

OS-ZZK-SUG-12 Public indexed-array API panics on an out-of-range predecessor index 92

OS-ZZK-SUG-13 Deposits fail when Token-2022 assesses a positive transfer fee 94

A. Vulnerability rating scale 96

B. Procedure 97

© 2026 Otter Audits LLC. All Rights Reserved.

# 1. Executive summary

## 1.1. Overview

Helius engaged OtterSec to conduct a security assessment of the Zolana ZK system. This assessment was conducted between August 10th and September 3rd, 2026. For information on our auditing methodology, refer to Appendix B.

## 1.2. Key findings

We produced 31 findings throughout this audit engagement.

Among the most impactful vulnerabilities, we identified that the P256 signature check relies on a scalar-multiplication gadget that never range-checks its hinted subscalars. A malicious prover can shift those values so that the bits the gadget actually uses come out as zero, which lets them forge P256 authorization for an input without the owner's private key (● OS-ZZK-ADV-00). We also found that the protocol configuration can be created by anyone when the program has no upgrade authority, or an unset one, at deployment time. A first caller could then name themselves protocol authority and permanently take over every administrative role, with no recovery path for the operator (● OS-ZZK-ADV-05).

Additionally, the deployment tooling creates the protocol's governance multisig with a threshold of one instead of the specified two-of-five, so a single compromised member key is enough to rotate the protocol authority and take over the rest of the hierarchy (● OS-ZZK-ADV-04). Furthermore, the private transaction hash is built from commitments with no added randomness. Anyone can test candidate commitments against it and link a published nullifier to the exact note it spends, which breaks the unlinkability the protocol promises (● OS-ZZK-ADV-03).

Moreover, an attacker who learns a pending nullifier can search offline for spend values that together cover its Bloom-filter positions and get them queued first. The victim's note then reads as already spent and stays frozen until the filter rotates, which can take a long time (● OS-ZZK-ADV-02).

We also showed that zero-value deposits are accepted and each one pushes a new root into the fixed 200-slot root history. An attacker can cheaply evict the roots that pending proofs depend on and force users to rebuild and resubmit their transactions (● OS-ZZK-ADV-08). In addition, merge proofs do not commit to the destination tree, so a caller can redirect a user's merged note to a different tree, which breaks wallet balance resolution and can delay later spends (● OS-ZZK-ADV-09).

We also suggested replacing the Poseidon injectivity axiom in the Lean proofs with a collision-resistance assumption. A function that compresses two field elements into one cannot be injective, so every hash-binding theorem built on that axiom is vacuous (● OS-ZZK-SUG-00). We further recommended that the tree account loader keep its mutable borrow guard alive for as long as it hands out writable tree data, since dropping the guard early allows aliased writes to the same account (● OS-ZZK-SUG-03). We also recommended checking that the root-history capacity recorded at tree initialization matches the number of slots the account layout actually holds, so nullifier retention

© 2026 Otter Audits LLC. All Rights Reserved.

assumptions cannot silently drift (OS-ZZK-SUG-04). Finally, we advised rejecting output commitments outside the scalar field with a proper program error instead of a panic (OS-ZZK-SUG-07), and adding length checks to the Merkle root gadget so that a mismatched height or path length cannot silently skip hashing levels (OS-ZZK-SUG-01).

© 2026 Otter Audits LLC. All Rights Reserved.

# 2. Scope

Zolana's ZK system uses Groth16 proofs over the BN254 curve to prove a private UTXO spend is valid without revealing amounts, assets, or (in policy rings) owners. Each proof shows the owner authorized the spend, the inputs exist in the on-chain tree, their nullifiers are unused (no double spends), and value is conserved per asset. The Solana program checks each proof against a single public hash it re-builds from the transaction itself, so a proof can't be reused for a different transaction. Policy rings also hide owners, add compliance proofs such as auditor encryption, and let a merge service consolidate small UTXOs without the owner signing.

The source code was delivered to us in a Git repository at github.com/helius-labs/zolana. This audit was performed against commit `c53d799`.

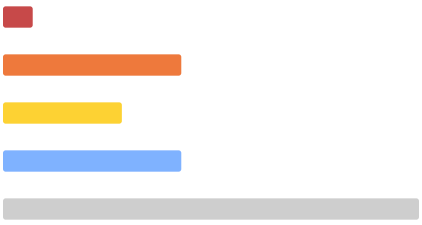
© 2026 Otter Audits LLC. All Rights Reserved.

# 3. Findings

Findings were rated using the criteria in Appendix A. We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

## 3.1. Findings summary

| Severity | Count |
|---|---|
| Critical | 1 |
| High | 6 |
| Medium | 4 |
| Low | 6 |
| Informational | 14 |



© 2026 Otter Audits LLC. All Rights Reserved.

# 4. Advisories

We identified 17 vulnerabilities during our audit. Advisories have **immediate** security implications, and we recommend remediating them as soon as possible.

| ID | Severity | Status | Description |
|---|---|---|---|
| OS-ZZK-ADV-00 | Critical | ✓ Resolved | Unbounded fake-GLV hints may bypass P256 ECDSA authorization |
| OS-ZZK-ADV-01 | High | ○ Ack'd | Unbounded prover HTTP intake permits resource exhaustion |
| OS-ZZK-ADV-02 | High | ✓ Resolved | Adaptive Bloom-filter coverage can selectively block a known nullifier |
| OS-ZZK-ADV-03 | High | ✓ Resolved | Deterministic private transaction hash exposes spent UTXO commitments |
| OS-ZZK-ADV-04 | High | ✓ Resolved | Incorrect threshold in 2-of-5 policy |
| OS-ZZK-ADV-05 | High | ✓ Resolved | Skipped protocol-config gate modes permit first-writer role capture |
| OS-ZZK-ADV-06 | High | ○ Ack'd | RingEddsa exact-intent authorization depends on trusted ring-program forwarding |
| OS-ZZK-ADV-07 | Medium | ✓ Resolved | Permissioned ring creation exposes the governance signer to candidate code |
| OS-ZZK-ADV-08 | Medium | ✓ Resolved | Cheap zero-value deposits can rotate the finite state-root history |
| OS-ZZK-ADV-09 | Medium | ✓ Resolved | Merge proof does not bind its output tree |
| OS-ZZK-ADV-10 | Medium | ○ Ack'd | The ring RPC cannot refresh an asset registry larger than 4,096 entries |
| OS-ZZK-ADV-11 | Low | ✓ Resolved | Batch address-append circuit trusts inconsistent batch size and slice length |
| OS-ZZK-ADV-12 | Low | ✓ Resolved | TypeScript tree layout constants omit the 64-byte reserved header |
| OS-ZZK-ADV-13 | Low | ✓ Resolved | Cached tree-update replay policy conflicts |
| OS-ZZK-ADV-14 | Low | ✓ Resolved | Address inputs bypass the dummy-input reserve gate |
| OS-ZZK-ADV-15 | Low | ✓ Resolved | Mixed default- and policy-ring P256 inputs reveal the shared owner |
| OS-ZZK-ADV-16 | Low | ✓ Resolved | Duplicate UTXO commitments make Photon return ambiguous membership proofs |

© 2026 Otter Audits LLC. All Rights Reserved.

## Unbounded fake-GLV hints may bypass P256 ECDSA authorization

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-00 | ● Critical | ✓ Resolved |

### Description

The scalar-multiplication invariant is that a hinted point $R$ must equal $[s]Q$. gnark's fake-GLV check is intended to obtain signed subscalars $s_1, s_2 < 2^{128}$ satisfying $s_1 + ss_2 \equiv 0 \bmod n$ and then establish the equivalent point relation $[s_1]Q + [s_2]R = \mathcal{O}$. Zolana's prover module pins `github.com/consensys/gnark` v0.15.0, so the generated RingP256 constraint systems inherit that release's fake-GLV scalar-multiplication constraints. In `scalarMulFakeGLV`, gnark obtains hinted subscalars $s_1$ and $s_2$, constrains their signed modular relation to the original scalar, and rejects a zero signed $s_2$. However, it does not constrain either hinted value to the approximately 128-bit range assumed by the subsequent accumulator.

The accumulator decomposes $s_1$ and $s_2$ but consumes only the lower `nbits`, where `nbits` is half the scalar-modulus bit length. Given an honest half-GCD result $(a, b)$, a malicious prover can instead supply values shifted by $2^{128}$. The modular relation and nonzero check still hold, while the consumed low 128 bits are zero. This underconstraint reaches the affected RingP256 authorization circuit through gnark's fake-GLV scalar-multiplication gadget. `p256Authorization` constrains the supplied P256 public key to the curve, derives its owner hash, reconstructs the public message scalar, and delegates signature verification to gnark's ECDSA `IsValid` gadget. Consequently, the underconstraint affects the circuit value intended to represent P-256 signature validity.

prover/server/circuits/spp_transaction/custom/p256.go

```
func (c *CustomRingP256Circuit) p256Authorization(
  api frontend.API,
) (frontend.Variable, frontend.Variable, frontend.Variable, error) {
  curve, err := sw_emulated.New[emulated.P256Fp, emulated.P256Fr](
    api,
    sw_emulated.GetCurveParams[emulated.P256Fp](),
  )
  if err != nil {
    return nil, nil, nil, err
  }
  point := sw_emulated.AffinePoint[emulated.P256Fp](c.Private.P256Pub)
  curve.AssertIsOnCurve(&point)
  [...]
  message := fr.FromBits(messageBits...)
  p256MessageHash := hashBytes32Bits(api, messageBits)
  p256SignatureValid := c.Private.P256Pub.IsValid(
    api,
    sw_emulated.GetCurveParams[emulated.P256Fp](),
    message,
    &c.Private.P256Sig,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
  )
  return p256PkHash, p256MessageHash, p256SignatureValid, nil
}
```

For each content-bearing input identified as P256-owned, `P256Signers` requires the shared `p256SignatureValid` value to hold. Consequently, weakening that value's ECDSA calculation weakens the circuit's authorization condition for every real P256 input, while the remaining transaction constraints continue to apply.

prover/server/circuits/spp_transaction/shared/signers.go

```
func P256Signers([...]) (Signers, []frontend.Variable) {
  [...]
  for i, in := range inputs {
    carriesContent := in.isUtxoOrAddress(api)
    isP256 := api.IsZero(ownerPkHashes[i])
    assertZeroWhen(api, api.Sub(1, carriesContent), ownerPkHashes[i])
    pkHash := api.Select(isP256, p256PkHash, ownerPkHashes[i])
    AssertWhen(api, carriesContent, api.Sub(1, api.IsZero(pkHash)))
    AssertWhen(api, api.Mul(carriesContent, isP256), p256SignatureValid)
    AssertWhen(
      api,
      api.Mul(carriesContent, api.Sub(1, isP256)),
      authorizedEddsa.Contains(api, ownerPkHashes[i]),
    )
    hasP256Input = api.Add(hasP256Input, api.Mul(carriesContent, isP256))
    signers[i] = api.Mul(carriesContent, pkHash)
    resolvedOwnerPkHashes[i] = pkHash
  }
  // RingP256 is the mixed P256/Ed25519 rail, not an alternative all-Ed25519
  // selector. All-Ed25519 transactions use RingEddsa.
  api.AssertIsDifferent(hasP256Input, 0)
  return signers, resolvedOwnerPkHashes
}
```

The ECDSA gadget computes the verification point from $[r/s]\ \mathrm{PK}$ and $[m/s]G$ ($q = [m/s]G + [r/s]\ \mathrm{PK}$) and accepts when $q_x = r$. Both scalar multiplications therefore provide independent opportunities to substitute attacker-selected hinted result points while satisfying the deficient fake-GLV checks. Complete-arithmetic mode separately detects a zero scalar and substitutes a nonzero value for the fake-GLV check. This creates a distinct $m \equiv 0 \bmod n$ case because the corresponding multiplication result is subsequently replaced with the point at infinity.

© 2026 Otter Audits LLC. All Rights Reserved.

Consensys/gnark/std/algebra/emulated/sw_emulated/point.go

```
func (c *Curve[B, S]) scalarMulFakeGLV(Q *AffinePoint[B], s
*emulated.Element[S], opts ...algopts.AlgebraOption) *AffinePoint[B] {
  [...]
  var isScalarZero frontend.Variable
  _s := s
  if !cfg.IncompleteArithmetic {
    isScalarZero = c.scalarApi.IsZero(s)
    _s = c.scalarApi.Select(isScalarZero, c.scalarApi.One(), s)
  }[...]
}
```

`scalarMulFakeGLV` asserts equality between an accumulator containing the two subscalar terms and a $[3]R$ bias and $[3]R$ as shown below. If only the low bits of both shifted subscalars are consumed, that equality degenerates and no longer binds the hinted result $R$ to $[s]Q$.

Consensys/gnark/std/algebra/emulated/sw_emulated/point.go

```
func (c *Curve[B, S]) scalarMulFakeGLV(Q *AffinePoint[B], s
*emulated.Element[S], opts ...algopts.AlgebraOption) *AffinePoint[B] {
  [...]
  if !cfg.IncompleteArithmetic {
    Acc = c.Select(c.api.Or(isScalarZero, isInputPointAtInfinity), tableR[2],
    Acc)
  }
  // we added [3]R at the last iteration so the result should be
  //    Acc = [s1]Q + [s2]R + [3]R
  //        = [s1]Q + [s2*s]Q + [3]R
  //        = [s1+s2*s]Q + [3]R
  //        = [0]Q + [3]R
  //        = [3]R
  c.AssertIsEqual(Acc, tableR[2])
  res := &AffinePoint[B]{
    X: *R[0],
    Y: *R[1],
  }
  if !cfg.IncompleteArithmetic {
    zero := c.baseApi.Zero()
    // On the complete-arithmetic edge branches (s=0 or Q=(0,0)) the
    // accumulator check is rerouted through a dummy path, so we must also
    // return the canonical infinity point instead of the unverified hint
    output.
    res = c.Select(c.api.Or(isScalarZero, isInputPointAtInfinity),
    &AffinePoint[B]{X: *zero, Y: *zero}, res)
  }
  return res
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
25 }
```

A malicious prover possessing a live input witness could choose on-curve multiplication-result hints that make an otherwise invalid P256 signature satisfy the circuit. If a vulnerable proving key and matching accepted verifying key are deployed and the required ring-policy and runtime conditions are met, this could authorize a transaction spending the exposed P256 input to attacker-selected outputs. Merkle inclusion, nullifier derivation and non-inclusion, balance conservation, output hashes, the public-input hash, eligible ring forwarding, and on-chain account checks must still succeed. The result applies only to the proof-bound transaction. It does not recover the owner's signing key, produce a reusable ECDSA signature, or affect Ed25519-only paths.

### Proof of concept

1. A user sends an intended RingP256 request to a malicious or compromised prover, disclosing the signature and private input data.
2. The prover replaces the outputs or public settlement with an attacker destination and recomputes the transaction hashes. The user's signature is invalid for this altered digest.
3. For both ECDSA multiplications, it supplies the shifted half-GCD values and chosen on-curve result points. Merkle inclusion, nullifier derivation and non-inclusion, balance conservation, output hashes, and the public-input hash must still hold.
4. It proves with a vulnerable proving key whose matching verifying key SPP accepts.
5. The attacker signs as payer and submits through an eligible ring program. SPP requires an active, signing ring-config account, so ring-policy forwarding, live roots, an unspent nullifier, and valid accounts remain runtime gates.
6. SPP verifies the proof, consumes the victim nullifier, appends the attacker output, and performs any proof-bound settlement. The victim's later spend fails as already spent.

### Remediation

Disable RingP256 proof acceptance until corrected artifacts are active. Pin gnark to a reviewed release or commit that constrains both fake-GLV half-GCD outputs to the bit range consumed by the accumulator, including the fix identified by upstream commit `a3ad59ad`. Regenerate and atomically rotate every affected P256 constraint system, proving key, BSB22 verifying key, circuit fingerprint, and proving-key lock entry, and ensure old selectors and verifying keys can no longer accept proofs.

### Patch

Resolved in fcc1c46.

© 2026 Otter Audits LLC. All Rights Reserved.

## Unbounded prover HTTP intake permits resource exhaustion

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-01 | High | Ack'd |

### Description

The prover buffers attacker-controlled proof requests without limiting their size or how long they take to read. `proveHandler.ServeHTTP` reads the whole POST body with `io.ReadAll` before `ParseProofRequestMeta` checks the circuit type or anything else about the request. The handler does not check `Content-Length` or wrap the body in `http.MaxBytesReader`, so the application puts no byte limit on either declared-length or chunked bodies, and both can use up memory.

prover/server/server/server.go

```
424 func (handler proveHandler) ServeHTTP(w http.ResponseWriter, r *http.Request)
    {
425   if r.Method != http.MethodPost {
426     w.WriteHeader(http.StatusMethodNotAllowed)
427     return
428   }
429
430   buf, err := io.ReadAll(r.Body)
431   if err != nil {
432     logging.Logger().Error().Err(err).Msg("Error reading request body")
433     malformedBodyError(err).send(w)
434     return
435   }
436
437   proofRequestMeta, err := common.ParseProofRequestMeta(buf)
438   if err != nil {
439     malformedBodyError(err).send(w)
440     return
441   }
442   [...]
443 }
```

With Redis configured, `/queue/add` does the same thing. It reads the full body before it parses metadata, picks a queue, deduplicates the request, or admits it. The prover also creates its `http.Server` with only `Addr` and `Handler` set, so `ReadHeaderTimeout`, `ReadTimeout`, and `IdleTimeout` are all zero. That means the application sets no deadline for incomplete headers, slow bodies, or idle keep-alive connections. Go's default limit on header size still applies, so the problem is the unbounded body size and read time.

Authentication only helps against the body-based attack when `PROVER_API_KEY` is set. In `authMiddleware`, an empty key lets every request through. A nonempty key has to be sent with the request before the route handler runs. Headers, though, are processed before any HTTP middleware

© 2026 Otter Audits LLC. All Rights Reserved.

gets to authenticate the request. Once the listener is reachable, slow or incomplete headers tie up connection resources without needing an API key at all.

prover/server/server/auth.go

```
45  func (m *authMiddleware) isAuthenticated(r *http.Request) bool {
46    if m.apiKey == "" {
47      return true
48    }
49
50    providedKey := m.extractAPIKey(r)
51    if providedKey == "" {
52      return false
53    }
54
55    return subtle.ConstantTimeCompare([]byte(m.apiKey), []byte(providedKey)) == 1
56  }
```

A client can send most of a large body and hold back the last byte or the terminating chunk. `io.ReadAll` then keeps everything it has received while it waits for EOF. Doing this over many connections at once drives up memory, file descriptor, goroutine, and connection usage before the handler ever gets a chance to reject bad metadata. Depending on how much capacity the deployment has, the result can be slower proofs, rejected legitimate requests, exhausted connections, or the prover process being killed for running out of memory. If external infrastructure closes these connections, keeping the denial going requires large, concurrent, or repeatedly replaced requests.

Whether this is reachable depends on the deployment. The CLI binds the prover to `0.0.0.0:5000` by default. The root Docker Compose configuration binds the published prover port to host loopback by default but lets an environment variable expose it to other machines. Binding to loopback reduces remote exposure, but it does nothing about untrusted local clients and does not limit request resources. A mandatory ingress that enforces body limits, deadlines, connection limits, and quotas could contain the issue, but the repository doesn't define the production ingress policy.

## Remediation

Write one shared, bounded body-reading routine and use it in both `/prove` and `/queue/add`. Wrap each request body in `http.MaxBytesReader`. Base the limit on the largest valid serialized request across all supported circuit shapes, plus documented headroom, and return HTTP 413 when the read fails with `*http.MaxBytesError`. An early `Content-Length` check can reject oversized requests up front, but it can't replace the bounded reader, since chunked requests don't send that header. On the prover server, set finite values for `ReadHeaderTimeout`, `ReadTimeout` (or an equivalent per-handler deadline for reading the body), `WriteTimeout`, and `IdleTimeout`, and set an explicit `MaxHeaderBytes` policy. Enforce limits on connections, byte rate, concurrent requests, and per-principal usage, either in the service or at a mandatory ingress.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Adaptive Bloom-filter coverage can selectively block a known nullifier

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-02 | High | ✓ Resolved |

### Description

Each note has a fixed nullifier derived from its commitment, blinding, and nullifier secret. Changing transaction-level parameters such as the payer, recipient, proof randomness, outputs, encryption salt, output tree, indexer, or RPC provider does not change that nullifier or its Bloom-filter probe positions. Pending-nullifier admission needs to reject only nullifiers that the queue holds. Instead, `Batch::insert` requires the candidate to be absent according to both live Bloom filters before adding it to the current hash chain and filter.

program-libs/batched-merkle-tree/src/batch.rs

```rust
pub fn insert<const NUM_ITERS: usize, const BYTES: usize>(
    &mut self,
    bloom_filter_value: &[u8; 32],
    hash_chain_value: &[u8; 32],
    bloom_filters: &mut [BloomFilter<NUM_ITERS, BYTES>; 2],
    hash_chain_store: &mut [[u8; 32]],
    bloom_filter_index: usize,
) -> Result<(), BatchedMerkleTreeError> {
    let other_bloom_filter_index = if bloom_filter_index == 0 { 1 } else { 0 }
    // 1. Pure checks first: the value must be in neither bloom filter.
    Self::check_non_inclusion(
        bloom_filter_value,
        bloom_filters
            .get(other_bloom_filter_index)
            .ok_or(BatchedMerkleTreeError::InvalidBatchIndex)?,
    )?;
    Self::check_non_inclusion(
        bloom_filter_value,
        bloom_filters
            .get(bloom_filter_index)
            .ok_or(BatchedMerkleTreeError::InvalidBatchIndex)?,
    )?;
    [...]
}
```

`Batch::check_non_inclusion` treats any positive `BloomFilter::contains` result as `NonInclusionCheckFailed`. Because the queue performs no exact pending-set lookup after that probabilistic result, it rejects an absent nullifier whenever other queued nullifiers have already set all of its probe bits.

© 2026 Otter Audits LLC. All Rights Reserved.

program-libs/batched-merkle-tree/src/batch.rs

```rust
pub fn check_non_inclusion<const NUM_ITERS: usize, const BYTES: usize>(
    value: &[u8; 32],
    bloom_filter: &BloomFilter<NUM_ITERS, BYTES>,
) -> Result<(), BatchedMerkleTreeError> {
    if bloom_filter.contains(value) {
        return Err(BatchedMerkleTreeError::NonInclusionCheckFailed);
    }
    Ok(())
}
```

The nullifier tree uses two fixed-size Bloom-filter generations configured for batches of 30,000 entries, with 4,603,072 bits and ten probes per value. The documented false-positive probability of $10^{-12}$ characterizes accidental collisions for ordinary inputs. It does not stop an adversary from searching for valid nullifiers whose probes cover a chosen target.

program-libs/batched-merkle-tree/src/constants.rs

```rust
pub const DEFAULT_ADDRESS_BATCH_SIZE: u64 = 30000;

pub const DEFAULT_ADDRESS_ZKP_BATCH_SIZE: u64 = 250;

// False positive probability 1.0E-12 for 30k elements.
pub const ADDRESS_BLOOM_FILTER_CAPACITY: u64 = 4_603_072;
pub const ADDRESS_BLOOM_FILTER_NUM_HASHES: u64 = 10;
```

The Bloom-filter probe function derives an initial position and stride from public, unkeyed Keccak output. Anyone who learns a victim's nullifier can calculate its ten positions and search offline for other valid nullifiers that set those positions.

program-libs/bloom-filter/src/lib.rs

```rust
/// Double-hash probe parameters for `value`.
///
/// `h2` (the stride) is derived from an independent hash slice and forced odd:
/// an even stride shares the power-of-two factor of the table size
/// (`num_bits` has factor 2^6 for the nullifier tree), confining all probes to
/// one congruence class; a zero stride (mod `num_bits`) would collapse every
/// probe onto a single bit. An odd stride only shares the table's odd factor,
/// and two decorrelated halves make `h1`/`h2` independent.
#[inline(always)]
fn probes(value: &[u8; 32]) -> (u64, u64) {
    let hash = solana_nostd_keccak::hash(value);
    let h1 = u64::from_le_bytes(hash[0..8].try_into().unwrap());
    let h2 = u64::from_le_bytes(hash[16..24].try_into().unwrap()) | 1;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
59      (h1, h2)
60  }
```

The configured indexer, a hosted prover, transaction-submission infrastructure, or a landed but failed transaction can disclose the target nullifier before a successful spend. If the queue admits a covering set ahead of the victim, the victim's unchanged nullifier fails queue insertion even though it is absent from both the exact nullifier tree and the attacker's queued values. This blocks spending of the affected note without enabling theft, forgery, or a double spend. Recovery depends on filter rotation and forester progress. Ten cover entries need another 29,990 insertions to fill that generation and 15,000 entries in the next generation before the clearing condition holds, so stalled traffic or forester processing can leave the denial in place with no fixed end.

## Proof of concept

1. Obtain the nullifier of a still-unspent funded note through one of the identified disclosure channels and compute its ten deterministic Bloom-filter positions.
2. Vary address-input seeds offline and retain candidates that satisfy the relevant constraints, set a target position not yet covered, and remain negative against both live filters.
3. Continue until ten distinct address-input nullifiers cover every target position, then arrange them as two supported five-input, three-output transactions using zero-value dummy outputs.
4. Submit both ahead of the victim. Each transaction must pass account checks and proof verification. On success, the per-input loop inserts every nullifier into the shared queue and maps insertion errors to `NullifierTreeUpdateFailed`.
5. If the queue admitted the covering transactions, the target remains absent from the exact nullifier tree and from the attacker's inserted values, but `contains` should report a positive result and queue insertion should fail with the mapped nullifier-tree update error.

## Remediation

Replace the pending Bloom filters with exact per-nullifier records so admission tests the identity of each queued nullifier. Before mutating state, check every instruction nullifier against the exact pending set and against the other nullifiers in the same instruction; if all checks succeed, update the exact set and queue metadata atomically. Existing trees require an explicit migration strategy because their Bloom-filter bits cannot enumerate pending nullifiers: reconstruct the exact pending set from authenticated queue data or prevent new admissions until the legacy generations have been processed and drained. Add regression coverage that first queues the demonstrated ten-value cover and verifies that the absent target remains admissible, while true duplicates from either live generation and duplicates within one instruction are rejected.

## Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Deterministic private transaction hash exposes spent UTXO commitments

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-03 | High | ✓ Resolved |

### Description

Zolana represents private balances as UTXO commitments in a public Merkle tree and publishes a nullifier when an owner spends a UTXO. The intended privacy property is that the nullifier does not reveal which commitment it consumes. The specification's privacy guarantees state this unlinkability.

The shared `PrivateTxHashCircuit` undermines this property. It hashes the input-commitment chain, output-commitment chain, address-commitment chain, and `ExternalDataHash` with no randomness. The preimage contains no fresh secret that an observer cannot reproduce. For each input slot, `constrainInput` contributes the UTXO commitment only when the slot is a real UTXO, contributes the commitment to the address chain only when it is an address, and otherwise contributes zero. An ordinary one-input UTXO spend has a single candidate commitment in its input chain and zero in its address chain.

prover/server/circuits/spp_transaction/shared/inputs.go

```
1  func constrainInput(api frontend.API, in Input, signals PublicInputUtxoInputs)
   (frontend.Variable, frontend.Variable) {
2    [...]
3    // UTXO checks:
4    // 1. UTXO hash must exist in state Merkle tree.
5    {
6      AssertWhen(api, isUtxo, in.checkInclusion(api, utxoHash,
       signals.UtxoTreeRoot))
7    }
8    // Dummy checks:
9    // 1. All UTXO fields and nullifier secret 0, except the blinding.
10   {
11     AssertWhen(api, in.isDummy(api), in.Utxo.CheckDummy(api))
12     assertZeroWhen(api, in.isDummy(api), in.NullifierSecret)
13   }
14   // Address checks:
15   // 1. All UTXO fields and nullifier secret 0, except the blinding and owner.
16   AssertWhen(api, isAddress, in.checkAddress(api))
17
18   // Only UTXOs and addresses must be accessible as such
19   // in zk program proofs via private transaction hash.
20   inputHash := api.Select(isUtxo, utxoHash, frontend.Variable(0))
21   addressHash := api.Select(isAddress, utxoHash, frontend.Variable(0))
22   return inputHash, addressHash
23 }
```

© 2026 Otter Audits LLC. All Rights Reserved.

`ConstrainOutput` verifies every supplied output commitment but contributes it to the private transaction hash only for a real output. A dummy output contributes zero. Because the transaction publishes all output commitments, an observer can enumerate the real-or-zero choice for each output slot. This requires at most $2^M$ masks for $M$ output slots.

prover/server/circuits/spp_transaction/shared/outputs.go

```
24 func ConstrainOutput(api frontend.API, utxo UtxoCircuitFields, hash, ownerSigned
   frontend.Variable) frontend.Variable {
25   isUtxo := utxo.isUtxo(api)
26   api.AssertIsEqual(api.Add(isUtxo, utxo.isDummy(api)), 1)
27
28   // Same asset-0 rule as the input side: a real output must name a real asset.
29   assertZeroWhen(api, isUtxo, api.IsZero(utxo.Asset))
30
31   // 1. All fields must be 0 except blinding.
32   AssertWhen(api, utxo.isDummy(api), utxo.CheckDummy(api))
33
34   // 2. if utxo program data is set owner must have signed.
35   dataIsSet := api.Sub(1, api.IsZero(utxo.DataHash))
36   AssertWhen(api, api.Mul(isUtxo, dataIsSet), ownerSigned)
37
38   utxoHash := UtxoHashCircuit(api, utxo)
39   api.AssertIsEqual(utxoHash, hash)
40
41   return api.Select(isUtxo, utxoHash, frontend.Variable(0))
42 }
```

After processing the inputs and outputs, `Transaction.Constrain` recomputes the private transaction hash using these selected chains and requires it to equal the public `PrivateTxHash`. This equality turns the public hash into a deterministic test for candidate input commitments.

prover/server/circuits/spp_transaction/shared/transaction.go

```
1  func (t Transaction) Constrain(api frontend.API, signers Signers, outputSigned
   []frontend.Variable) error {
2    [...]
3    // 4. Check private tx hash.
4    privateTxHash := PrivateTxHashCircuit(
5      api,
6      inputHashes,
7      outputHashes,
8      addressHashes,
9      t.ExternalDataHash,
10   )
11   api.AssertIsEqual(privateTxHash, t.PrivateTxHash)
12   [...]
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
13 }
```

The `TransactIxData` instruction publishes `private_tx_hash`, the output commitments, the encryption salt, the viewing key, the interface transfers, and the other data incorporated into `external_data_hash`. The salt is transaction-specific but public, so it cannot serve as private blinding against the commitment oracle. The on-chain `process_transact_ix` path resolves the public transfers and outputs, computes `external_data_hash` from instruction fields and account-derived values, assigns it to the proof inputs, and verifies the proof. A passive observer with the instruction and its accounts can reproduce the same external hash without submitting a transaction, controlling a prover, or learning a UTXO preimage.

programs/shielded-pool/src/instructions/transact/processor.rs

```
pub fn process_transact_ix([...]) -> ProgramResult {
    [...]
    let resolved_interface_transfers =
        resolve_interface_transfers(&ix, &transact_accounts.settlements);

    let external_data_hash = ExternalDataHash {
        spp_instruction_discriminator: instruction as u8,
        expiry_unix_ts: ix.expiry_unix_ts,
        interface_transfers: &resolved_interface_transfers,
        data_hash: ix.data_hash,
        ring_data_hash: ix.ring_data_hash,
        tx_viewing_pk: ix.tx_viewing_pk,
        salt: ix.salt,
        outputs: &resolved_outputs,
        messages: &ix.messages,
    }
    .hash()
    .map_err(|_| ShieldedPoolError::TransactProofVerificationFailed)?;
    proof_inputs.assign_external_data_hash(external_data_hash);
    proof_inputs.ensure_complete()?;

    TransactProof::new(&ix, &proof_inputs).verify()?;
    [...]
}
```

The shared relation is used by the confidential Ed25519, ring Ed25519, ring P256, and ring-authority transaction families, making applicable one-input transactions vulnerable across these paths. For such transactions, an observer can substitute each historical Merkle-tree commitment into the known formula, enumerate any output masks, and compare each result with the published `private_tx_hash`. A match identifies the consumed commitment value unless duplicate commitment values make the corresponding leaf position ambiguous, which exposes an edge between the transaction that created the commitment and the transaction that spent it. Repeated recovery may trace one-input paths and

© 2026 Otter Audits LLC. All Rights Reserved.

can weaken sender anonymity when linked activity includes a public deposit, withdrawal, default-ring identity, or relevant off-chain record.

### Remediation

Add a fresh, high-entropy private field element to the `private_tx_hash` preimage and ensure it is never included in transaction data, accounts, or emitted events.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Incorrect threshold in 2-of-5 policy

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-04 | High | ✓ Resolved |

### Description

The documented governance model assigns `protocol_authority` to an autonomous Squads smart account with a 2-of-5 threshold, while the tree-creation, ring-creation, and forester authorities are controlled 1-of-N accounts. Signer changes throughout the hierarchy also require a 2-of-5 protocol-authority transaction, as shown in the authority-governance specification. The subsequent `Sync execution` specification describes operators submitting `execute_transaction_sync_v2` with one key for threshold-one accounts. Applying this single-key flow to the protocol account would conflict with both 2-of-5 requirements in the specification. The initialization tool's `authorities::PROTOCOL` constant defines the five keys used as protocol members. The specification intends the two-signature quorum to operate over these members.

xtask/src/init_protocol.rs

```
pub mod authorities {
    use solana_pubkey::{pubkey, Pubkey};
    pub const PROTOCOL: [Pubkey; 5] = [
        pubkey!("2kgbLowvCQuMWxDKbHUZAURycziuRrvmtTuDEYMGMRsj"),
        pubkey!("AdWdKMo89o1HN2dMF1Bk9zRhtU7iT6tFPqL27uWoaMBi"),
        pubkey!("ESuhzg7TyJGBWToxxvsKez9HxP4KAKRDshBznppyRMDo"),
        pubkey!("GoZBYjLaMcjX1T6mqLBkeYehRDBb2ts19S2H6icvMBFd"),
        pubkey!("ECBkPzeojfxQpUGNM6u1dd1woER3wfmcYSVPbV8gxhJE"),
    ];
    [...]
}
```

In `create_all_smart_accounts`, the tool builds the protocol signer set from those five keys and passes it to `create_smart_account_with_retry` with no `settings_authority`. That makes the protocol account autonomous, as specified, but leaves its quorum to the shared helper.

xtask/src/init_protocol.rs

```
fn create_all_smart_accounts(
    rpc: &SolanaRpc,
    payer: &Keypair,
    treasury: &Pubkey,
) -> Result<[RoleAddrs; 5]> {
    let protocol_signers = signer_set(&authorities::PROTOCOL);
    let forester_signers = signer_set(&authorities::FORESTER);
    let protocol =
        create_smart_account_with_retry(rpc, payer, treasury, None,
        &protocol_signers, "protocol")?;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    let protocol_vault = protocol.vault;
    [...]
}
```

That shared helper grants every supplied member all permissions and calls `create_smart_account_ix` with `threshold` one and `time_lock` zero for every role. The client serializes those values into `CreateSmartAccountArgs` with no protocol-specific correction. The initialized protocol account is autonomous but operates as 1-of-5 instead of the specified 2-of-5. The resume and reuse paths verify only that the expected member keys are present and that the settings address matches its stored seed. They do not validate the threshold, time lock, settings authority, complete member set, or permission masks. The shielded-pool program does not add another quorum checka and its protocol-authority loader requires only that the supplied authority account is a signer and that its address equals the configured `protocol_authority` (the design expects Squads to validate membership and threshold before signing the inner CPI with the vault PDA). The checked-in protocol-update command also constructs its synchronous Squads execution with one protocol member. The stored threshold of one lowers the effective governance boundary.

Thus, an attacker holding one member key of an affected protocol Settings account may authorize an update that changes `ProtocolConfig.protocol_authority` to an attacker-controlled signing address, provided the new address also signs as the rotation instruction requires. Once the state points to that address, the attacker may act without the original Squads account to rotate the tree-creation, forester, and ring-creation authorities, modify the permissionless-creation flags, or pause trees.

### Proof of concept

1. The attacker obtains one member key from a protocol Settings account created by the affected initialization path. No second protocol-member key is required because the stored threshold is one.
2. They then construct an `UpdateProtocolConfigData::ProtocolAuthority` update naming an attacker-controlled signing address `A`, including it as the required new-authority signer, and wraps the update in `execute_transaction_sync_v2` with the compromised member as the sole threshold signer. The checked-in update command demonstrates the one-member synchronous execution pattern. Selecting the authority-rotation variant needs a custom client.
3. The compromised member has all permissions, one signature satisfies the stored threshold, and the zero time lock permits immediate execution. Squads then invokes the shielded-pool program with the current vault PDA as signer.
4. The shielded-pool program observes a signing authority whose address matches the current `protocol_authority`, verifies that `A` signed and matches the requested new address, and stores `A` as the new protocol authority.
5. `ProtocolConfig.protocol_authority` now equals `A`, and further governance no longer depends on continued access to the compromised member or the original Squads account..

### Remediation

Make the smart-account threshold, time lock, settings authority, member set, and permissions explicit role-specific inputs. Create the autonomous protocol account with exactly the five approved members, threshold two, time lock zero, no settings authority, and the intended protocol-member permissions;

© 2026 Otter Audits LLC. All Rights Reserved.

retain threshold one only for controlled operator accounts. Replace protocol-vault uses of single-key `execute_transaction_sync_v2` in initialization and maintenance tooling with a proposal, two-distinct-member approval, and execution flow, because changing only the creation threshold would make the existing one-member operations fail. Extend Settings decoding and validation in the resume and reuse paths to verify the complete policy: `settings_authority`, threshold, time lock, exact member set, and each member's permission mask. Inventory deployed protocol Settings accounts and their creation transactions. Migrate any 1-of-5 protocol vault by raising its threshold or rotating to a new 2-of-5 vault through a documented two-operator ceremony.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Skipped protocol-config gate modes permit first-writer role capture

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-05 | High | ✓ Resolved |

### Description

The canonical `ProtocolConfig` PDA is a one-time authority root for protocol administration, tree creation, forester operations, ring creation, and the associated permissionless-creation flags. As shown in `process_create_protocol_config`, the fee payer must sign and equal the submitted `protocol_authority`, but this equality is self-authentication because the caller supplies both identities. The processor then calls `check_initialization_authority`, creates the canonical PDA, and writes every submitted authority and flag into it.

programs/shielded-pool/src/instructions/protocol_config/create.rs

```rust
pub fn process_create_protocol_config(accounts: &mut [AccountView], data: &[u8])
-> ProgramResult {
    let data = *bytemuck::try_from_bytes::<CreateProtocolConfigData>(data)
        .map_err(|_| ShieldedPoolError::InvalidInstructionData)?;
    let mut iter = AccountIterator::new(accounts);
    let fee_payer = iter.next_signer("fee_payer")?;
    let protocol_config = iter.next_mut("protocol_config")?;
    let system_program = iter.next_account("system_program")?;
    let program = iter.next_account("program")?;
    let program_data = iter.next_account("program_data")?;

    if !pinocchio_system::check_id(system_program.address()) {
        return Err(ProgramError::IncorrectProgramId);
    }
    if *fee_payer.address() != data.protocol_authority {
        return Err(ShieldedPoolError::UnauthorizedCaller.into());
    }
    check_initialization_authority(fee_payer, program, program_data)?;

    let bump = verify_pda(
        protocol_config.address(),
        &[SPP_PROTOCOL_CONFIG_PDA_SEED],
        &crate::ID,
    )?;

    CreatePdaAccount {[...]}
    .execute()
    .map_err(|_| ShieldedPoolError::InvalidProtocolConfig)?;
    ProtocolConfigInitParams {[...]}
    .init(protocol_config)
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

The independent initializer binding is conditional. For a loader-v3 program whose genuine `ProgramData` account contains a present, nonzero upgrade authority, `check_initialization_authority` requires the fee payer to equal that authority. It rejects an incorrect program address and, on the loader-v3 path, an incorrect `ProgramData` address or owner and malformed loader state. It returns success for a non-v3 program owner and for loader-v3 `ProgramData` containing an unset or all-zero authority.

If the canonical configuration is still absent while one of these carve-outs is reachable, an unrelated caller can fund the first creation, name its own signing key as `protocol_authority`, and choose the remaining authorities and flags. Successful initialization creates and stamps the singleton, while a subsequent creation cannot overwrite it. Only `process_update_protocol_config` can change the resulting configuration, and it validates the supplied signer against the stored protocol authority before applying any update, as shown below.

programs/shielded-pool/src/instructions/protocol_config/update.rs

```
pub fn process_update_protocol_config(accounts: &mut [AccountView], data: &[u8])
-> ProgramResult {
    let data = UpdateProtocolConfigData::try_from_slice(data)
        .map_err(|_| ShieldedPoolError::InvalidInstructionData)?;
    let mut iter = AccountIterator::new(accounts);
    let authority = iter.next_signer("authority")?;
    let protocol_config = iter.next_mut("protocol_config")?;

    if let UpdateProtocolConfigData::ProtocolAuthority(authority) = &data {
        let new_authority = iter.next_signer("new_authority")?;
        if !address_eq(new_authority.address(), authority) {
            return Err(ShieldedPoolError::InvalidInstructionData.into());
        }
    }
    [...]
}
```

Protocol-authority rotation does not provide an operator-controlled recovery path. The same `process_update_protocol_config` path requires the current stored protocol authority to authorize every mutation and requires the incoming authority to co-sign a protocol-authority rotation. An attacker that completed the first write can retain control or rotate the configuration, while the intended initializer cannot recover the canonical account through an ordinary update.

A successful race captures administrative control of the deployment address. Subject to each downstream instruction's remaining checks, the attacker may control configured creation and forester roles, pause or unpause trees through the protocol-authority path, and permit or obstruct administrative resource creation.

© 2026 Otter Audits LLC. All Rights Reserved.

### Proof of concept

1. An operator publicly exposes the executable shielded-pool program before its canonical `ProtocolConfig` PDA has been initialized, with the genuine deployment selecting the non-v3, unset-upgrade-authority, or zero-upgrade-authority branch.
2. Have an unrelated funded signer submit `create_protocol_config` using its own address as both fee payer and `protocol_authority`, while selecting the other authorities and permissionless-creation flags and supplying the genuine program and applicable loader accounts.
3. The signature, payload-equality, System Program, program-identity, and canonical-PDA checks pass. For a non-v3 deployment the loader binding is skipped. For an unset or zero loader-v3 authority, the genuine `ProgramData` relationship and ownership checks pass but no nonzero authority is available for comparison.
4. The program creates the canonical PDA and initializes its fields from the attacker's payload because the new account has the expected size and zeroed data.
5. A later legitimate creation attempt fails because the singleton already exists. Updating or rotating the captured configuration requires authorization from the attacker's stored `protocol_authority`, with a new authority also required to co-sign a rotation.

### Remediation

Retain the existing comparison against a present, nonzero loader-v3 upgrade authority, but fail closed whenever initialization cannot derive an independently authenticated initializer. This includes non-v3 deployments and loader-v3 states with an unset or zero authority. If those deployment modes must be supported, require a one-time bootstrap permit signed by a precommitted key and bind it to the cluster, program ID, canonical config PDA, every authority and permissionless flag, and a replay nonce. Deployment tooling should initialize and verify the canonical configuration before finalization, migration, or public release, and record the loader state and initialization transaction.

### Patch

Resolved in 8bc92342d.

© 2026 Otter Audits LLC. All Rights Reserved.

## RingEddsa exact-intent authorization depends on trusted ring-program forwarding

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-06 | High | Ack'd |

### Description

Holding a RingEddsa input witness, including `OwnerPkHash` and `NullifierSecret`, does not suffice to spend the input. For every content-bearing input, `AuthorizedEddsaInputOwners` requires the private owner identity to be nonzero and present in the authorized signer vector. The on-chain program constructs that vector from the signing payer followed by the contiguous run of runtime signer accounts, so an unsigned victim account does not contribute its identity.

prover/server/circuits/spp_transaction/shared/signers.go

```
40  // AuthorizedEddsaInputOwners resolves the private per-input owner identities
    and
41  // requires every content-bearing input to be authorized by the unified signer
42  // vector. The vector contains the payer first, then appended Solana signers.
43  func AuthorizedEddsaInputOwners(
44    api frontend.API,
45    inputs []Input,
46    ownerPkHashes []frontend.Variable,
47    authorized Signers,
48  ) Signers {
49    owners := EddsaOnlySigners(api, inputs, ownerPkHashes)
50    for i, in := range inputs {
51      carriesContent := in.isUtxoOrAddress(api)
52      assertZeroWhen(api, api.Sub(1, carriesContent), ownerPkHashes[i])
53      AssertWhen(api, carriesContent, authorized.Contains(api, ownerPkHashes[i]))
54    }
55    return owners
56  }
```

The `constrainInput` ownership check then derives the nullifier public key from the private nullifier secret, combines it with the selected signer identity, and requires the resulting owner hash to equal the owner committed in the UTXO. Retaining the victim identity while only the attacker signs fails signer membership and substituting the attacker identity fails the committed-owner comparison. Neither path yields a valid spend of the victim UTXO.

prover/server/circuits/spp_transaction/shared/inputs.go

```
1  func constrainInput(api frontend.API, in Input, signals PublicInputUtxoInputs)
   (frontend.Variable, frontend.Variable) {
2    [...]
3    // Checks UTXO and address:
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    // 1. Check owner hash matches UTXO.
    {
      nullifierPk := abstractor.Call(api, nullifierPkGadget{
        NullifierSecret: in.NullifierSecret,
      })
      ownerHash := abstractor.Call(api, ownerHashGadget{
        OwnerKeyHash: signals.SignerPk,
        NullifierPk:  nullifierPk,
      })
      ownerIsCorrect := api.IsZero(api.Sub(ownerHash, in.Utxo.Owner))
      AssertWhen(api, in.isUtxoOrAddress(api), ownerIsCorrect)
    }
    [...]
  }
```

The narrower authorization gap arises at the CPI trust boundary. A victim's Solana signature authorizes an outer instruction to a ring program, while the shielded-pool program accepts signer privilege propagated by that program and verifies the inner SPP payload. It does not require the outer instruction bytes the victim authorized to equal the inner payload the ring program constructs. The proof's public inputs bind the signer vector, ring identity, external data, nullifiers, outputs, roots, settlements, and expiry, but that binding authenticates only the payload submitted to SPP.

The attack does not work from witness possession alone, so it has no demonstrated unauthorized-spend impact under Solana's standard invoked-program trust model. However, under a hostile-ring containment model, malicious or compromised active ring code could replace the apparent transaction payload while forwarding the victim's signer privilege and supplying its ring-auth PDA signature. If an attacker also possesses the victim's complete witness and can prepare a valid, unexpired proof for current roots, SPP may accept the replacement transaction, queue the victim input's nullifier, and append attacker-controlled output commitments.

### Proof of concept

1. Obtain the complete witness and nullifier secret for a victim UTXO, then prepare a valid, unexpired RingEddsa proof using current roots and attacker-controlled outputs.
2. Bind an active ring configuration to malicious or compromised ring code, and induce the victim to sign an outer call that supplies the required trees and the victim account as the payer or an owner signer.
3. Have the ring program disregard the apparent outer payload, preserve the victim's propagated signer privilege, sign with its ring-auth PDA, and invoke SPP using the prepared replacement payload.
4. SPP derives an authorized signer set containing the victim identity and verifies the replacement proof. Because the proof is valid for the substituted inner transaction, SPP can queue the victim UTXO's nullifier and append the attacker-controlled output commitment without bypassing the circuit constraints.

© 2026 Otter Audits LLC. All Rights Reserved.

### Remediation

Document that RingEddsa owner authorization relies on runtime signer privilege and that an invoked ring program is trusted to preserve or validate the complete SPP intent. Wallets should allow-list trusted ring program identities and display the final tree addresses, settlements, recipients, amounts, outputs, and expiry before requesting a signature.

### Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Permissioned ring creation exposes the governance signer to candidate code

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-07 | Medium | ✓ Resolved |

### Description

In permissioned mode, ring registration does not bind the configured ring-creation authority's approval to the complete initial configuration. `process_create_ring_config` verifies the canonical `ring_auth` PDA for the submitted `program_id`, funds that account from the payer, and then initializes `authority` and `ring_authority_transact_is_enabled` from the CPI instruction data. The payer's authorization establishes which ring program is registered but does not authenticate the other initial policy fields, as shown in `process_create_ring_config`.

programs/shielded-pool/src/instructions/ring_config/create.rs

```
pub fn process_create_ring_config(accounts: &mut [AccountView], data: &[u8]) ->
ProgramResult {
    [...]
    {
        let protocol = load_protocol_config(protocol_config)?;
        if !protocol.allows_permissionless_ring_creation()
            && protocol
                .check_ring_creation_authority(payer.address())
                .is_err()
        {
            return Err(ShieldedPoolError::UnauthorizedCaller.into());
        }
    }
    pinocchio_system::create_account_with_minimum_balance([...])
    .map_err(|_| ShieldedPoolError::InvalidRingConfig)?;

    RingConfigInitParams {
        authority: data.authority,
        program_id: data.program_id,
        ring_authority_transact_is_enabled:
        data.ring_authority_transact_is_enabled,
        bump: ring_auth_bump,
    }
    .init(ring_config)
}
```

The required call topology exposes this gap. The payer must sign and, when ring creation is not permissionless, match `protocol_config.ring_creation_authority`. The `ring_config` account must also be the canonical PDA derived under the candidate program and carry signer privilege. Because only that candidate program can provide the PDA signature through a signed CPI, the configured authority cannot complete registration by calling SPP itself. The candidate receives a signed,

© 2026 Otter Audits LLC. All Rights Reserved.

writable payer account while keeping control over the inner CPI data, despite the checks shown in `process_create_ring_config`.

The test client demonstrates this account topology in `create_ring_config`, where it invokes the ring program with the payer as a writable signer, the canonical `ring_auth` account writable, and both the System Program and SPP available. The repository's test ring program forwards the received instruction data and account privileges while adding the PDA signature, but that verbatim forwarding is fixture behavior. SPP enforces no such invariant. A malicious candidate may keep its `program_id` while replacing the submitted authority or enabled flag.

sdk-libs/program-test/src/ring.rs

```rust
pub fn create_ring_config([...]) -> Result<Pubkey, ProgramTestError> {
    let ring_program = Self::ring_test_program_id();
    // The config account IS the ring's canonical `ring_auth` PDA.
    let (ring_config, _) = pda::ring_auth(&ring_program);
    let data = CreateRingConfigData {
        program_id: RING_TEST_PROGRAM_ID.into(),
        authority: authority.to_bytes().into(),
        ring_authority_transact_is_enabled,
    };
    let ix = Instruction {
        program_id: ring_program,
        accounts: vec![
            AccountMeta::new(payer.pubkey(), true),
            AccountMeta::new_readonly(pda::protocol_config(), false),
            AccountMeta::new(ring_config, false),
            AccountMeta::new_readonly(Pubkey::default(), false),
            AccountMeta::new_readonly(self.program_id, false),
        ],
        data: encode_instruction(tag::CREATE_RING_CONFIG, &data),
    };
    self.send(&[ix], &[payer])?;
    Ok(ring_config)
}
```

The specification places `create_ring_config` in the ring-program administrative interface, consistent with registration occurring through candidate-controlled code. As reflected by that interface, governance approval of an outer ring-program invocation does not itself prove that the inner SPP invocation contains the same initial configuration.

A malicious candidate may register its own ring with an authority and authority-transaction policy that governance never approved. Subsequent update and ownership-transfer processing authenticates the stored ring authority, so the substituted authority may change the enabled and paused flags or participate in authority rotation, while the intended authority has no administrative access. The canonical PDA

© 2026 Otter Audits LLC. All Rights Reserved.

validation confines the issue to the candidate's own ring and does not permit modification of another ring's configuration or the protocol configuration.

The writable payer also creates a deployment-dependent SOL-loss path. Candidate-controlled code may skip SPP, transfer lamports from the writable, System-owned payer to the uninitialized `ring_auth` address, and later use the PDA signature to transfer the parked SOL onward while the address remains System-owned. This impact depends on production forwarding a funded vault and is bounded by its available SOL.

### Remediation

Implement two-phase registration. First, require governance to call SPP or a trusted registrar itself to create a single-use approval that commits to `program_id`, `authority`, `ring_authority_transact_is_enabled`, an expiry, and every other initial policy field. Then permit the candidate program to create its canonical `ring_auth` configuration only when the CPI fields exactly match that approval, consuming the approval atomically. Separate authorization from funding by passing the ring-creation authority read-only and using an unrelated writable payer for rent.

### Patch

Resolved in 8bc92342d.

© 2026 Otter Audits LLC. All Rights Reserved.

## Cheap zero-value deposits can rotate the finite state-root history

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-08 | Medium | ✓ Resolved |

### Description

The UTXO tree stores historical roots in a fixed array of 200 physical slots. Shielded transaction and merge instructions identify a root by its slot index, and the proof path loads whatever value occupies that slot. A proof built against root `R` at `root_history[i]` stays usable only while slot `i` still contains `R`, as the fixed-capacity layout reflects.

program-libs/tree/src/smt.rs

```rust
pub const ROOT_HISTORY_CAPACITY: usize = 200;

#[repr(C)]
#[derive(Clone, Copy)]
pub struct UtxoTreeLayout<const HEIGHT: usize> {
    pub next_index: [u8; 8],
    pub root: [u8; 32],
    pub root_history_cursor: [u8; 2],
    pub root_history_len: [u8; 2],
    pub subtrees_len: u8,
    pub subtrees: [[u8; 32]; HEIGHT],
    pub root_history_capacity: u8,
    pub root_history: [[u8; 32]; ROOT_HISTORY_CAPACITY],
}
```

The `append_batch` function calculates roots while adding leaves but records only the final root of a nonempty batch. Each successful batch advances the history once, however many leaves it contains.

program-libs/tree/src/smt.rs

```rust
pub fn append_batch<'l, I>(&mut self, leaves: I) -> Result<(), TreeError>
where
    I: IntoIterator<Item = &'l [u8; 32]>,
{
    let zero_bytes = Poseidon::zero_bytes();
    let mut leaves = leaves.into_iter().peekable();
    while let Some(leaf) = leaves.next() {
        [...]
        // Only the batch-final root enters the history. Pushing per-leaf
        // placeholders would evict real roots batch_size times faster and
        // `root_by_index` rejects zero slots, so they are never usable.
        if is_last {
            self.root = current_level_hash;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
            self.push_root(current_level_hash);
        }
        self.set_next_index(self.next_index() + 1);
    }
    Ok(())
}
```

The `push_root` function advances the cursor modulo the 200-slot capacity and writes the new root into the resulting slot. A root that is current when a proof is built gets overwritten after 200 further successful append batches, and a root already partway through that window gets overwritten sooner. The mechanism does not distinguish updates by Solana slot, so multiple successful batches within the same slot consume multiple history entries.

program-libs/tree/src/smt.rs

```
fn push_root(&mut self, root: [u8; 32]) {
    let capacity = self.root_history.len();
    if capacity == 0 {
        return;
    }
    let cursor = usize::from(self.current_root_index());
    let len = usize::from(u16::from_le_bytes(self.root_history_len));
    let next = (cursor + 1) % capacity;
    let next_len = (len + 1).min(capacity);
    if let Some(slot) = self.root_history.get_mut(next) {
        *slot = root;
    }
    self.root_history_cursor = (next as u16).to_le_bytes();
    self.root_history_len = (next_len as u16).to_le_bytes();
}
```

A proofless deposit can drive this rotation without transferring value. In `process_deposit_internal`, the amount enters the UTXO commitment without a nonzero check, and the resulting commitment goes to `append_batch`. The surrounding settlement logic executes the asset transfer only when the aggregate amount exceeds zero, while it still emits the output and deposit event.

programs/shielded-pool/src/instructions/deposit/processor.rs

```
fn process_deposit_internal<'a, const HAS_RING: bool>([...]) -> ProgramResult {
    [...]
    let mut tree =
        TreeAccount::from_account_view_mut(parsed.tree, &crate::ID,
        TREE_ACCOUNT_DISCRIMINATOR)
            .map_err(ShieldedPoolError::from)?;
    let first_output_leaf_index = tree.utxo_tree().next_index();

```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    // One batch append: only the last leaf hashes up to the root, so a batch
    // costs one root recomputation instead of one per entry.
    tree.utxo_tree()
        .append_batch(utxo_hashes.iter())
        .map_err(ShieldedPoolError::from)?;
    drop(tree);
    [...]
}
```

Repeated accepted append batches can replace the slot an already-built proof references, so a later transaction using that proof fails because verification reads the slot's newer root. An affected user may need to fetch updated Merkle data, generate a new proof, sign a replacement transaction, and pay for another submission. However, this behavior does not corrupt the tree, erase the underlying note, validate a false proof, or move the user's funds.

## Remediation

Define the minimum retention period promised for UTXO roots under adversarial append traffic. If no retention guarantee is intended, document that the 200th later append batch can replace a just-recorded root and require clients to detect stale-root failures, fetch a retained root, regenerate the proof, re-sign, and retry.

## Patch

Resolved in 8bc92342d.

© 2026 Otter Audits LLC. All Rights Reserved.

## Merge proof does not bind its output tree

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-09 | Medium | ✓ Resolved |

### Description

Merge proof verification does not bind the output commitment to the tree that receives it. `MergeProof::public_input_hash` folds in the nullifiers, output commitment, roots derived from the input tree, private transaction hash, external-data hash, dummy-input policy, and the applicable owner or ring binding, but neither the output-tree address nor its state. Replacing destination tree A with tree B leaves the public input for an otherwise unchanged proof the same.

programs/shielded-pool/src/instructions/merge/verify.rs

```
85  pub fn public_input_hash(&self) -> Result<[u8; 32], ProgramError> {
86      let nullifiers = create_hash_chain_from_slice(&self.ix.nullifiers)?;
87      let utxo_roots = create_hash_chain_from_slice(&self.derived.utxo_roots)?;
88      let nullifier_tree_roots =
89          create_hash_chain_from_slice(&self.derived.nullifier_tree_roots)?
90
91      let prefix = [
92          nullifiers,
93          *self.ix.output_utxo_hash,
94          utxo_roots,
95          nullifier_tree_roots,
96          *self.ix.private_tx_hash,
97          self.derived.external_data_hash,
98          self.derived.allow_dummy_inputs,
99      ];
100
101     let prefix_hash = create_hash_chain_from_slice(&prefix)?
102     match &self.derived.owner_binding {
103         MergeOwnerBinding::Ring {
104             ring_program_id,
105             output_ring_data_hash,
106         } => create_hash_chain_from_slice(&[
107             prefix_hash,
108             *output_ring_data_hash,
109             *ring_program_id,
110         ])
111         .map_err(Into::into),
112         MergeOwnerBinding::Registry { signing_pk_field } => {
113             create_hash_chain_from_slice(&[prefix_hash,
                *signing_pk_field]).map_err(Into::into)
114         }
115     }
116 }
```

© 2026 Otter Audits LLC. All Rights Reserved.

The nested `MergeExternalDataHash` does not provide the missing destination binding. Its preimage contains only the merge instruction discriminator, expiry, and output commitment, so it remains identical when the payer-signed instruction substitutes a different output tree.

program-libs/interface/src/instruction/instruction_data/merge_transact.rs

```rust
pub struct MergeExternalDataHash<'a> {
    pub spp_instruction_discriminator: u8,
    pub expiry_unix_ts: u64,
    pub output_utxo_hash: &'a [u8; 32],
}

impl MergeExternalDataHash<'_> {
    pub fn hash(&self) -> Result<[u8; 32], HasherError> {
        let mut preimage = Vec::new();
        preimage.push(self.spp_instruction_discriminator);
        preimage.extend_from_slice(&self.expiry_unix_ts.to_be_bytes());
        preimage.extend_from_slice(self.output_utxo_hash);
        Sha256BE::hash(&preimage)
    }
}
```

On the default merge route, the owner's registry record must have merging enabled, after which any caller may initiate the merge and the payer is the only required instruction signer. `MergeTransactAccounts::validate_and_parse` accepts separate writable input and output trees, while `process_merge_core` derives roots and the dummy-input policy from the input tree before loading the supplied output tree and appending the commitment there. The SDK follows the same separation, where it obtains proofs for the specified input tree and builds the proof before placing the caller-supplied `output_tree` into the instruction.

The `merge_ring` route has the same output-tree omission in its proof inputs, but its account boundary also requires the active ring configuration and payer to sign. Redirection through this route requires the ring program to authorize a caller-selected destination, a precondition the current code does not establish.

programs/shielded-pool/src/instructions/merge_ring/account.rs

```rust
impl<'a> MergeRingAccounts<'a> {
    pub fn validate_and_parse(accounts: &'a mut [AccountView]) -> Result<Self,
    ProgramError> {
        let mut iter = AccountIterator::new(accounts);
        let input_tree = iter.next_mut("input_tree")?;
        let output_tree = iter.next_mut("output_tree")?;
        let ring_config = iter.next_signer("ring_config")?;
        let ring_program_id = load_active_ring_config(ring_config)?.program_id;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        let payer = iter.next_signer("payer")?;
        let system_program = iter.next_account("system_program")?;
        if !pinocchio_system::check_id(system_program.address()) {
            return Err(ShieldedPoolError::InvalidSystemProgram.into());
        }
        Ok(Self {
            input_tree,
            output_tree,
            payer,
            ring_program_id,
        })
    }
}
```

Redirection to a healthy, indexed tree does not destroy the resulting note because the merge event records the actual output tree and assigned leaf index, so the wallet may reconstruct the owner-indexed output. It still disrupts the high-level wallet when another eligible note for the same asset remains on the original tree. The wallet's spend-tree resolver returns `AmbiguousTree` when eligible unspent holdings span multiple trees.

programs/shielded-pool/src/instructions/merge/event.rs

```
pub fn build_merge_event(
    ix: &MergeTransactIxDataRef<'_>,
    tree_write: MergeTreeWrite,
    output_view_tag: [u8; 32],
    output_data: Vec<u8>,
) -> GeneralEvent {
    [..]
    GeneralEvent {
        inputs: tree_write.inputs,
        outputs,
        messages: Vec::new(),
        tx_viewing_pk: [0u8; 33],
        salt: [0u8; 16],
        first_output_leaf_index: tree_write.output_leaf_index,
        output_tree: tree_write.output_tree,
        spl_transfers: Vec::new(),
    }
}
```

The availability impact grows when the substituted tree accepts the output append but its nullifier queue is awaiting maintenance, because a later spend may fail during queue insertion until a forester processes the batch. A paused or UTXO-full destination rejects the merge and rolls back the instruc-

© 2026 Otter Audits LLC. All Rights Reserved.

tion. Each successful redirection requires a payer-funded merge transaction. The omission does not enable theft, ownership changes, inflation, proof forgery, or an on-chain double spend.

### Remediation

Add a canonical encoding of the actual output-tree address to `MergeExternalDataHash`, and compute that hash the same way in the SDK and on-chain program. Also enforce an independent destination policy because a service holding the proving inputs can generate a fresh proof bound to a chosen tree. Policy-ring programs should validate the destination before signing the CPI.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## The ring RPC cannot refresh an asset registry larger than 4,096 entries

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-10 | Medium | Ack'd |

### Description

This issue concerns a later-revision integration defect. The Ring RPC service code at the audited commit `c53d7998` does not contain it. The affected implementation first appeared in commit `93ebf405`. It remains relevant because it consumes registry state that the on-chain program at the audited revision can create. The protocol requires the audit service to resolve every valid on-chain asset registry entry a ring transaction uses, and the specification identifies the Ring RPC as the component that holds the auditor key and serves decrypted transaction data.

The later `asset_registry` implementation retrieves all shielded-pool accounts whose data length equals the 48-byte `SplAssetRegistry::SIZE`. It rejects the complete response when the RPC returns more than `MAX_ASSET_REGISTRY_ACCOUNTS`, or 4,096, accounts, and it counts before decoding account discriminators. Thus, 4,096 entries succeed, while 4,097 entries deterministically fail. Counting before decoding also implies any future 48-byte SPP account type would consume the same allowance.

services/ring-rpc/src/upstream.rs

```
async fn asset_registry(&self) -> Result<AssetRegistry, ClientError> {
    let program = Address::new_from_array(SHIELDED_POOL_PROGRAM_ID);
    let accounts = self
        .rpc.client()
        .get_program_ui_accounts_with_config(
            &program,
            RpcProgramAccountsConfig {
                filters: Some(vec![RpcFilterType::DataSize(SplAssetRegistry::SIZE as u64)]),
                account_config: RpcAccountInfoConfig {
                    encoding: Some(UiAccountEncoding::Base64),
                    commitment: Some(CommitmentConfig::confirmed()),
                    ..RpcAccountInfoConfig::default()
                },
                ..RpcProgramAccountsConfig::default()
            },
        ).await
        .map_err(|_| ClientError::Rpc("asset registry request failed".to_owned()))?;
    if accounts.len() > MAX_ASSET_REGISTRY_ACCOUNTS {
        return Err(ClientError::Rpc("asset registry response is too large".to_owned(),));
    }
    let entries = accounts.into_iter().filter_map(|(_, account)| {
        let account = account.to_account()?;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        SplAssetRegistry::from_account_bytes(&account.data).ok()
            .map(|registry| (registry.asset_id, registry.mint))
    });
    AssetRegistry::new(entries).map_err(ClientError::from)
}
```

The on-chain `SplAssetCounter::allocate_id` function does not impose a corresponding 4,096-entry limit. It begins assigning SPL asset identifiers at 2 and advances the counter using checked `u64` addition, allowing the registry to grow beyond the later service's boundary.

program-libs/interface/src/state/spl_asset_counter.rs

```
impl SplAssetCounter {
    [...]
    pub const FIRST_ASSET_ID: u64 = 2;
    [...]
    pub fn allocate_id(&mut self) -> Result<u64, InterfaceError> {
        let id = self.next_id;
        if id < Self::FIRST_ASSET_ID {
            return Err(InterfaceError::InvalidDiscriminator);
        }
        self.next_id = id
            .checked_add(1)
            .ok_or(InterfaceError::InvalidDiscriminator)?;
        Ok(id)
    }
}
```

In `process_create_spl_interface`, each successful registration validates the mint, requires either permissionless creation or the protocol authority, allocates the next asset identifier, and creates a distinct registry PDA and token vault funded by the signing authority. When permissionless registration is enabled, or when the caller controls the protocol authority, a caller can register enough distinct compatible mints to push the on-chain registry into the unsupported 4,097-entry state.

A fresh Ring RPC initializes its `AssetRegistry` with only the SOL mapping, as shown by the registry's `Default` implementation. During an authorized audit read, an uncached SPL asset makes the service refresh the registry. If the complete query contains more than 4,096 matching accounts, the refresh error propagates from the read without replacing the cached registry, and the failed-refresh state prevents another attempt for 60 seconds.

sdk-libs/transaction/src/wallet/asset.rs

```
impl Default for AssetRegistry {
    fn default() -> Self {
        Self(HashMap::from([(SOL_ASSET_ID, SOL_MINT)]))
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
66          }
67      }
```

This produces sustained audit-read unavailability for pages containing uncached assets: after each backoff interval, the same oversized query fails again until the implementation or on-chain state changes. A cold restart suffers most because the first audited SPL transaction requires a refresh, whereas a warm process may keep serving pages that use only cached assets. The failure is limited to off-chain audit reads, and exploitation requires thousands of successful registrations and their account-funding costs.

## Remediation

For the later Ring RPC integration, ensure that the service can synchronize every registry state the on-chain program admits. Prefer an indexed, slot-consistent source that provides a bounded initial snapshot followed by account notifications; validate each record's discriminator and contents, and replace the cache only after completing a coherent update. If synchronization must retain a fixed full-response cap, enforce the identical maximum in `create_spl_interface` before allocating an asset identifier or creating either account, and document that protocol limit. Also, do not turn one unknown asset or failed refresh into failure of an otherwise valid page. Preserve cached results, mark only the affected transaction with a stale-registry status, and schedule refresh on its own timer.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Batch address-append circuit trusts inconsistent batch size and slice length

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-11 | Low | ✓ Resolved |

### Description

For batch size `B` and tree height `H`, the circuit template must contain exactly `B` entries in each value, index, and proof vector, and each proof row must contain exactly `H` siblings. This ensures that the same `NewElementValues` determine both the indexed-tree transition and the queue hash-chain commitment. The protocol specification identifies `batch_update_nullifier_tree` as the instruction that inserts queued nullifiers through this batch proof, as shown in the instruction table.

The `BatchAddressTreeAppendCircuit.Define` transition loop processes indices from zero through `BatchSize - 1`. Each iteration accesses the corresponding low-element data, new-element value, and Merkle-proof rows, but `Define` does not first verify that the six outer slices contain exactly `BatchSize` entries or that each proof row contains exactly `TreeHeight` siblings. After the transition, `Define` computes the hash chain over the complete `NewElementValues` slice with no `BatchSize` limit. A template containing more than `BatchSize` new values may commit to elements that take no part in the Merkle transition, while an undersized slice or proof row may make circuit compilation fail.

prover/server/circuits/nullifier_tree.go

```
1  func (circuit *BatchAddressTreeAppendCircuit) Define(api frontend.API) error {
2    [...]
3    api.AssertIsEqual(circuit.NewRoot, currentRoot)
4
5    leavesHashChain := gadget.HashChain(api, circuit.NewElementValues)
6    api.AssertIsEqual(circuit.HashchainHash, leavesHashChain)
7
8    publicInputsHashChain := circuit.computePublicInputHash(api)
9    api.AssertIsEqual(circuit.PublicInputHash, publicInputsHashChain)
10
11   return nil
12 }
```

`BatchSize`, `TreeHeight`, and the slice lengths are host-side circuit-template parameters that determine the compiled constraint system. They are not witness values that a remote prover may alter after setup. The canonical constructor allocates the expected dimensions, the proof request path invokes `ValidateShape` before witness construction, and the production key manager recognizes only height 40 with batch size 10 or 250, as shown by `determineBatchKeyPath`.

prover/server/prover/common/lazy_key_manager.go

```
204 func (m *LazyKeyManager) determineBatchKeyPath(circuitType CircuitType,
    treeHeight uint32, batchSize uint32) string {
205   switch circuitType {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
206     case BatchAddressAppendCircuitType:
207       if treeHeight == 40 && batchSize == 250 {
208         return m.keyPath("batch_address-append_40_250.key")
209       } else if treeHeight == 40 && batchSize == 10 {
210         return m.keyPath("batch_address-append_40_10.key")
211       }
212     }
213
214     return ""
215   }
```

The impact is a circuit-construction contract gap. A maintainer or external in-process consumer could compile a malformed constraint system whose queue commitment and Merkle transition cover different numbers of elements, or trigger a compilation failure with undersized data. However, the production path exposes no untrusted-request exploit. Producing an accepted divergent transition would also require matching malformed proving and verifying artifacts, plus an integration that supplies the longer queue commitment, while the audited request and on-chain verification paths remain restricted to the canonical templates.

## Remediation

At the beginning of `BatchAddressTreeAppendCircuit.Define`, before emitting any constraints, return an error unless `LowElementValues`, `LowElementNextValues`, `LowElementIndices`, `LowElementProofs`, `NewElementValues`, and `NewElementProofs` each contain exactly `BatchSize` entries. Also require every row of `LowElementProofs` and `NewElementProofs` to contain exactly `TreeHeight` entries. Retain `BatchAddressAppendParameters.ValidateShape` as a separate request-boundary check.

## Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## TypeScript tree layout constants omit the 64-byte reserved header

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-12 | Low | ✓ Resolved |

### Description

The exported TypeScript tree-layout metadata does not match the layout Rust enforces. As shown by `TreeAccountLayout`, the `repr(C)` account header contains a one-byte discriminator, a one-byte state field, six bytes of padding, and a 64-byte reserved region before the UTXO tree. In the current implementation, both TypeScript values omit this reserved region and land 64 bytes early.

program-libs/tree/src/lib.rs

```rust
/// Bytes reserved in the account header for future tree metadata.
pub const TREE_RESERVED_BYTES: usize = 64;

#[repr(C)]
#[derive(Clone, Copy)]
pub struct TreeAccountLayout<
    const UTXO_HEIGHT: usize,
    const RH: usize,
    const NUM_ITERS: usize,
    const BLOOM: usize,
    const ZKP: usize,
> {
    pub discriminator: u8,
    pub state: u8,
    pub _padding: [u8; 6],
    pub _reserved: [u8; TREE_RESERVED_BYTES],
    pub utxo: UtxoTreeLayout<UTXO_HEIGHT>,
    pub nullifier: NullifierLayout<RH, NUM_ITERS, BLOOM, ZKP>,
}
```

Within `UtxoTreeLayout`, the eight-byte `next_index` field precedes the 32-byte current root, and `ROOT_OFFSET` is eight bytes. Because the UTXO tree begins after the 72-byte account header, the root begins at byte 80. The complete Rust layout occupies 1,185,728 bytes.

program-libs/tree/src/smt.rs

```rust
pub const ROOT_OFFSET: usize = 8;

pub const ROOT_HISTORY_CAPACITY: usize = 200;

#[repr(C)]
#[derive(Clone, Copy)]
pub struct UtxoTreeLayout<const HEIGHT: usize> {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    pub next_index: [u8; 8],
    pub root: [u8; 32],
    [...]
}
```

The TypeScript module defines `TREE_ACCOUNT_SIZE` as 1,185,664 and `STATE_ROOT_OFFSET` as 16. As shown by the interface barrel, the supported `@zolana/sdk/interface` entry point re-exports these constants, exposing the stale values to SDK consumers.

sdk-libs/ts/src/interface/index.ts

```
export type { Shape } from "./shape.js";
export {
  ADDRESS_TREE_HEIGHT,
  ADDRESS_TREE_INPUT_QUEUE_BATCH_SIZE,
  ADDRESS_TREE_INPUT_QUEUE_ZKP_BATCH_SIZE,
  ADDRESS_TREE_ROOT_HISTORY_CAPACITY,
  FIRST_ASSET_ID,
  FORESTER_REIMBURSEMENT_LAMPORTS,
  STATE_HEIGHT,
  STATE_ROOT_OFFSET,
  StateDiscriminator,
  TREE_ACCOUNT_SIZE,
  foresterFeePerQueueElement,
} from "./state.js";
```

The `process_create_tree` instruction receives an existing writable tree account and passes its data to `TreeAccount::init`. That initializer requires the buffer length to equal the Rust layout size, and the processor maps initialization failures to `InvalidTreeAccounts`, as shown below. The program rejects every account allocated with the exported TypeScript size. When allocation and initialization are separate, the undersized account may remain unusable.

programs/shielded-pool/src/instructions/create_tree.rs

```
pub fn process_create_tree(accounts: &mut [AccountView], data: &[u8]) ->
ProgramResult {
    [...]
    let tree_pubkey = tree.address().to_bytes();
    let mut tree_data = tree
        .try_borrow_mut()
        .map_err(|_| ProgramError::AccountBorrowFailed)?;

    TreeAccount::init(
        &mut tree_data,
        TREE_ACCOUNT_DISCRIMINATOR,
        STATE_HEIGHT as u8,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
            tree_pubkey,
            nullifier_params,
        )
        .map_err(|_| ShieldedPoolError::InvalidTreeAccounts)?;
        Ok(())
    }
```

Reading 32 bytes from the exported offset 16 lands inside the reserved header, not on the current root at bytes 80 through 111. Initialization zeroes the reserved region, so consumers may read an all-zero value and display a wrong root, lose synchronization, or waste work constructing a proof against the wrong root. This does not replace the verifier's root because transaction processing reads the selected root from the typed on-chain tree, and a proof using the incorrect root fails closed.

### Remediation

Generate the TypeScript `TREE_ACCOUNT_SIZE` and `STATE_ROOT_OFFSET` exports from the canonical Rust `tree_account_size()` and `state_root_offset()` accessors. The corrected values are 1,185,728 and 80, respectively.

### Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Cached tree-update replay policy conflicts

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-13 | Low | ✓ Resolved |

### Description

The occupied-slot behavior of `update_tree_from_address_queue` contradicts its documented contract. The helper delegates proof processing to `verify_proof_cache_update`. Its comments state that a proof targeting an already occupied slot returns `false` and becomes a no-op. However, the implementation does not enforce that condition..

program-libs/batched-merkle-tree/src/merkle_tree_update.rs

```
/// Verify one address-append proof and cache the update at its zkp batch
/// index. Returns `true` when a new update was cached, or `false` when the
/// update is already applied (its zkp batch index is behind the number of
/// inserted zkp batches) or already cached (an occupied slot at this
/// StartIndex exists); a replayed proof is then a no-op.
///
/// Steps:
/// 1. Validate the zkp batch index and that its hash chain is finalized.
/// 2. Return `false` if the update is already applied, then reconstruct the
///    proof's StartIndex, the tree next index this zkp batch writes at.
/// 3. Rebuild the public input hash and verify the proof.
/// 4. Store the cached update, keyed by StartIndex, at its zkp batch index.
 fn verify_proof_cache_update(
    &mut self,
    instruction_data: &InstructionDataAddressAppendInputs,
) -> Result<bool, BatchedMerkleTreeError> {
    [...]
}
```

Each `CachedTreeUpdate` contains an `old_root`, a `new_root`, and an `occupied` byte, and `is_occupied` exposes whether the slot contains a pending transition. This state allows proofs for future batches to remain cached until earlier batches advance the apply frontier, but the store path never consults the occupancy indicator before writing a verified transition into the selected slot.

program-libs/batched-merkle-tree/src/zero_copy.rs

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy, PartialEq, Eq, bytemuck::Pod,
bytemuck::Zeroable)]
pub struct CachedTreeUpdate {
    pub old_root: [u8; 32],
    pub new_root: [u8; 32],
    pub occupied: u8,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
74  }
75
76  impl CachedTreeUpdate {
77      pub fn is_occupied(&self) -> bool {
78          self.occupied != 0
79      }
80  }
```

For a submission at proof-batch index `j`, `verify_proof_cache_update` validates the physical batch and proof-batch bounds, requires the corresponding hash chain to be ready, and returns early if the tree has already applied `j`. It then derives the transition's start index, constructs the public input from the submitted roots, stored leaf hash chain, and start index, and verifies the Groth16 proof. After successful verification, it assigns the submitted roots and an occupied value to the selected cache entry without first checking that entry's current occupancy. Another valid proof for the same stored queue values and start index can replace an existing future transition; an invalid proof fails before assignment and replaces nothing.

When cached transitions reach the apply frontier, `apply_cached_tree_updates` applies a transition only if its `old_root` equals the tree's current root. The function clears a replacement whose old root does not match without appending its new root, and the correct transition must then be submitted again. This guard stops a proof for a noncurrent tree state from changing the live root, but a successful replacement can discard another authorized operator's cached proof and cause redundant verification, eviction, and resubmission work.

The audited regression also encodes first-write-wins behavior by expecting an occupied replay to return without verification and preserve the original entry, whereas the implementation reaches proof verification. The protocol specification does not specify whether occupied future slots should be immutable or replaceable, and repository history shows that an earlier occupancy guard was removed. Exploiting the replacement path requires a configured forester authority and repeated valid, out-of-order submissions. The ordinary forester submits indices in order, and a sole malicious forester may already interrupt maintenance by withholding updates.

## Remediation

Choose and document one occupied-slot policy, then align the implementation, comments, specification, and tests with that decision. If replacement is intended, retain proof verification before overwriting an occupied future slot, update the stale no-op comments and regression, and add proof-backed tests confirming that a valid corrected transition replaces a stale entry, an invalid replacement preserves the existing entry, and the apply-time root comparison evicts a stale transition. If first-write-wins is intended, check `CachedTreeUpdate::is_occupied` right after validating the target slot and before constructing public inputs or invoking proof verification, return `Ok(false)` for an occupied slot, and retain the apply-time old-root comparison and eviction. The first-write-wins design should document that a stale valid entry cannot be corrected until it reaches the frontier, fails the live-root comparison, and is cleared for resubmission.

© 2026 Otter Audits LLC. All Rights Reserved.

### Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Address inputs bypass the dummy-input reserve gate

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-14 | Low | ✓ Resolved |

### Description

`Transaction.Constrain` constrains `AllowDummyInputs` as a Boolean and applies it only to each input's dummy classification. When the flag is false, the gate requires `isDummy` to be false, but it does not require `isAddress` to be false. The subsequent `constrainInput` call treats spendable UTXOs, addresses, and padding dummies as distinct domains. Address inputs remain subject to their owner binding, zero-valued content fields, derived nullifier, nullifier non-inclusion, and transaction-hash binding. The circuit does not accept them as unchecked substitutes for spendable inputs.

prover/server/circuits/spp_transaction/shared/transaction.go

```
func (t Transaction) Constrain(api frontend.API, signers Signers, outputSigned
[]frontend.Variable) error {
  [...]
  api.AssertIsBoolean(t.AllowDummyInputs)
  // 1. check inputs
  inputHashes := make([]frontend.Variable, t.Shape.NInputs)
  addressHashes := make([]frontend.Variable, t.Shape.NInputs)
  for i, in := range t.Inputs {
    api.AssertIsEqual(
      api.Mul(api.Sub(1, t.AllowDummyInputs), in.isDummy(api)),
      0,
    )
    signals := PublicInputUtxoInputs{
      Nullifier:         t.Nullifiers[i],
      UtxoTreeRoot:      t.UtxoTreeRoots[i],
      NullifierTreeRoot: t.NullifierTreeRoots[i],
      SignerPk:          signers[i],
    }
    inputHashes[i], addressHashes[i] = constrainInput(api, in, signals)
  }
  [...]
}
```

The program obtains `allow_dummy_inputs` from the input tree before processing the instruction's inputs. `TreeAccount::allow_dummy_inputs` compares remaining nullifier queue capacity, including reservations, with unused UTXO-tree capacity. The loop then queues one nullifier for every input slot without distinguishing between spendable UTXOs, addresses, and padding dummies, so an address and a dummy each consume one nullifier-tree reservation.

programs/shielded-pool/src/instructions/transact/tree.rs

© 2026 Otter Audits LLC. All Rights Reserved.

```rust
#[profile]
pub(crate) fn apply_input_tree([...]) -> Result<InputTreeResult, ProgramError> {
    [...]
    let allow_dummy_inputs =
    bool_field(input_tree.allow_dummy_inputs().map_err(tree_error)?);
    let mut inputs = Vec::with_capacity(ix.inputs.len());
    let mut utxo_roots = [[0u8; 32]; MAX_INPUTS];
    let mut nullifier_tree_roots = [[0u8; 32]; MAX_INPUTS];
    let nullifier_seq_base =
    input_tree.nullifer_tree().queue_batches.next_index;
    for (i, input) in ix.inputs.iter().enumerate() {
        // 1. Assign state tree root to proof inputs.
        *utxo_roots.get_mut(i).ok_or(error)? = input_tree
            .get_utxo_tree_root(input.utxo_tree_root_index)
            .map_err(tree_error)?;
        // 2. Assign nullifier tree root to proof inputs.
        *nullifier_tree_roots.get_mut(i).ok_or(error)? = input_tree
            .get_nullifier_tree_root(input.nullifier_tree_root_index)
            .map_err(tree_error)?;
        // 3. insert_nullifier_into_queue
        input_tree
            .nullifer_tree()
            .insert_nullifier_into_queue(&input.nullifier_hash)
            .map_err(|_| ShieldedPoolError::NullifierTreeUpdateFailed)?;
        // 4. Build indexer nullifier queue data.
        inputs.push(Input {
            tree: input_tree_address,
            input_queue_seq: nullifier_seq_base + i as u64,
            nullifier: input.nullifier_hash,
        });
    }
    proof_inputs.assign_input_tree(
        &utxo_roots[..ix.inputs.len()],
        &nullifier_tree_roots[..ix.inputs.len()],
        allow_dummy_inputs,
    )?;[...]
}
```

This creates a policy asymmetry but does not violate the documented dummy-only invariant: when the flag is false, the circuit rejects padding dummies while valid, owner-authorized address inputs remain permitted. An address input can reduce the input tree's remaining nullifier capacity even when it is already below the tree's unused UTXO capacity, above all when outputs go to another tree. No proof, ownership, or nullifier-uniqueness check is bypassed. Exhausting the nullifier tree would also require each insertion to pass the tree's capacity checks and, from a fresh tree, a sustained difference of $2^{40} - 2^{32}$ nullifier reservations relative to UTXO appends.

© 2026 Otter Audits LLC. All Rights Reserved.

### Remediation

If the policy is intended to prohibit padding only, retain the circuit gate but rename `allow_dummy_inputs` to `allow_padding_dummy_inputs`, document that owner-authorized address inputs remain valid and consume nullifier queue capacity, monitor capacity using queued reservations, and add a regression showing that a valid address input remains provable when the flag is false. If the intended policy instead reserves capacity against every nonspending input, first define the required post-transaction capacity invariant, then apply the gate to both dummy and address domains and calculate it using every input reservation together with whether outputs append to the same tree.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Mixed default- and policy-ring P256 inputs reveal the shared owner

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-15 | Low | ✓ Resolved |

### Description

The RingP256 circuit preserves the owner tag of a policy-ring P256 input only when every real P256 input belongs to the policy ring. `P256Signers` treats a zero private owner field as the P256 selector and resolves every real P256 input to the same `p256PkHash`. `AssertDefaultP256Owner` then detects whether any real P256 UTXO or address input has a zero ring program ID and, if so, requires the public default-owner component to equal that shared hash. Because the check does not distinguish other P256 inputs in the same proof by domain, one default-ring input makes the identity shared with policy-ring inputs public.

prover/server/circuits/spp_transaction/shared/signers.go

```
94  // AssertDefaultP256Owner publishes the shared P256 identity iff at least one
95  // real P256 UTXO/address belongs to the default ring. Ring-only P256 inputs
    keep
96  // the shared owner private and require the public field to be zero.
97  func AssertDefaultP256Owner(
98    api frontend.API,
99    inputs []Input,
100   ownerPkHashes []frontend.Variable,
101   p256PkHash frontend.Variable,
102   defaultP256OwnerPkHash frontend.Variable,
103 ) {
104   hasDefaultP256 := frontend.Variable(0)
105   for i, in := range inputs {
106     carriesContent := in.isUtxoOrAddress(api)
107     isP256 := api.IsZero(ownerPkHashes[i])
108     isDefaultRing := api.IsZero(in.Utxo.RingProgramID)
109     hasDefaultP256 = api.Add(
110       hasDefaultP256,
111       api.Mul(carriesContent, isP256, isDefaultRing),
112     )
113   }
114   hasDefaultP256 = api.Sub(1, api.IsZero(hasDefaultP256))
115   api.AssertIsEqual(defaultP256OwnerPkHash, api.Mul(hasDefaultP256,
    p256PkHash))
116 }
```

The Rust proof builder places the raw x-coordinate of the shared P256 public key in `default_owner_tag` when a real default-ring P256 input is present and incorporates its hash into the proof input. The on-chain `public_input_hash` hashes the instruction-carried tag for RingP256

© 2026 Otter Audits LLC. All Rights Reserved.

proofs on its own. Omitting or substituting the tag changes the verifier's public input and invalidates the proof.

programs/shielded-pool/src/instructions/transact/verify.rs

```rust
pub fn public_input_hash(&self) -> Result<[u8; 32], ProgramError> {
    [...]
    if self.ix.circuit.is_p256() {
        let message_digest = Sha256::hash(self.ix.private_tx_hash)
            .map_err(|_| ShieldedPoolError::TransactProofVerificationFailed)?;
        fields.push(hash_bytes(&message_digest)?);
        fields.push(match self.ix.circuit.default_p256_owner_tag() {
            Some(owner_tag) => hash_bytes(owner_tag)?,
            None => [0u8; 32],
        });
    }
    fields.push(self.derived.external_data_hash);
    [...]
}
```

The `default_ring_p256_input_exposes_and_binds_owner_tag` integration test exercises this binding for a default-ring input: it submits a transfer with an incorrect tag and expects rejection, then submits one carrying the correct x-coordinate. This test confirms publication and proof binding for the default-ring case; it does not exercise a transaction that mixes default-ring and policy-ring P256 inputs.

program-tests/ring-test-program/tests/p256_ring_lifecycle.rs

```rust
    Ok(())
}
#[test]
#[serial]
fn default_ring_p256_input_exposes_and_binds_owner_tag() -> Result<()> {
    let mut harness = RingHarness::new()?;
    harness.create_enabled_ring_config()?;
    harness.make_p256_actor("piper")?;
    for _ in 0..2 {
        harness.shield_default_sol("piper", 1_000_000_000)?;
    }

    harness.ring_transfer_p256_wrong_default_owner_tag_rejected(
        "piper",
        "riley",
        SOL_MINT,
        300_000_000,
    )?;
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
79      harness.ring_transfer_p256_default_input_exposes_owner_tag(
80          "piper",
81          "riley",
82          SOL_MINT,
83          300_000_000,
84      )?;
85      harness.sync("riley")?;
86      Ok(())
87  }
```

If the same P256 key owns both a real default-ring input and a real policy-ring input in one proof, the default-ring input causes the shared parity-free P256 owner tag to appear on the wire. An observer with transaction-specific information identifying the policy-ring input could then link that activity to the same protocol identity. The public transaction alone establishes only that some real default-ring P256 UTXO or address input exists. It does not identify a particular policy-ring input or prove that another P256 input belongs to the policy ring. The owner must authorize the mixed input set, and this behavior does not disclose the private key, note opening, amount, asset, nullifier secret, or enable an unauthorized spend.

## Remediation

If this linkage is an accepted product tradeoff, document the P256 exception alongside the mixed-transaction privacy claim and warn users before signing a transaction that combines default-ring and policy-ring P256 inputs. If policy-ring owner unlinkability must hold across mixed transactions, compute separate `has_default_p256` and `has_policy_p256` indicators over real P256 UTXO and address inputs in the RingP256 circuit, and constrain their product to zero. Outputs and dummy input slots must not set either indicator. Mirror the same rejection in coin selection and proof construction to provide an early user-facing error.

## Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Duplicate UTXO commitments make Photon return ambiguous membership proofs

| ID | Severity | Status |
|---|---|---|
| OS-ZZK-ADV-16 | Low | ✓ Resolved |

### Description

The state tree and Photon use different identities for an output occurrence. The tree can hold the same commitment `C` at distinct indices `i` and `j`, but the supported Merkle-proof request identifies an output only by its tree and commitment. Photon resolves that request through the indexed output hash and rejects it when the hash occurs more than once, leaving neither occurrence addressable through the hash-only API. Proof-backed transactions permit this state because `Transaction.Constrain` requires input nullifiers to be distinct but constrains each output on its own without comparing output commitments. Repeating the full commitment preimage produces identical output hashes in separate output slots.

prover/server/circuits/spp_transaction/shared/transaction.go

```
1  func (t Transaction) Constrain(api frontend.API, signers Signers, outputSigned
   []frontend.Variable) error {
2    [...]
3    AssertDistinctNullifiers(api, t.Nullifiers)
4
5    // 2. check outputs
6    outputHashes := make([]frontend.Variable, t.Shape.NOutputs)
7    for i, utxo := range t.Outputs {
8      outputHashes[i] = ConstrainOutput(api, utxo, t.OutputHashes[i],
       outputSigned[i])
9    }
10
11   // 3. check balance
12   assertBalanceConservation(
13     api,
14     inputUtxos(t.Inputs),
15     t.Outputs,
16     t.PublicAssets[:],
17     t.PublicAmounts[:],
18   )
19   [...]
20 }
```

The on-chain `apply_output_tree` path records the index assigned to the first output and then appends every supplied output hash as a batch. It does not reject equal hashes, so equal commitments can occupy separate, otherwise valid tree positions.

© 2026 Otter Audits LLC. All Rights Reserved.

programs/shielded-pool/src/instructions/transact/tree.rs

```rust
#[profile]
pub(crate) fn apply_output_tree(
    output_tree_account: &mut AccountView,
    ix: &TransactIxDataRef<'_>,
    inputs: Vec<Input>,
) -> Result<TreeWrite, ProgramError> {
    [...]
    // Leaf index the first output lands at; the rest follow sequentially.
    let first_output_leaf_index = output_tree.utxo_tree().next_index();
    output_tree
        .utxo_tree()
        .append_batch(ix.outputs.iter().map(|o| o.utxo_hash))
        .map_err(tree_error)?;
    Ok(TreeWrite {
        inputs,
        first_output_leaf_index,
        output_tree: output_tree_address,
    })
}
```

The proofless default-ring deposit path recomputes each commitment from the public deposit fields and appends the resulting batch without a uniqueness check. An attacker can copy an unspent deposit whose event reveals the required preimage fields, but must supply the same asset and amount, pay transaction fees, target an active non-full tree, and act before the victim retains or obtains an exact-index proof. This duplication does not create value. As shown by `assertBalanceConservation`, each output occurrence adds to the per-asset output sum, while a proofless deposit settles the copied amount.

prover/server/circuits/spp_transaction/shared/balance.go

```go
func assertBalanceConservation([...]) {
  [...]
  assets := make([]frontend.Variable, 0,
  len(inputs)+len(outputs)+len(publicAssets))
  for _, input := range inputs {
    rangeCheck64(api, input.Amount)
    assets = append(assets, input.Asset)
  }
  for _, output := range outputs {
    rangeCheck64(api, output.Amount)
    assets = append(assets, output.Asset)
  }
  assets = append(assets, publicAssets...)

```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    // 4. For every asset sum in + public amount == sum out
    for _, asset := range assets {
      inSum := frontend.Variable(0)
      for _, input := range inputs {
        match := api.IsZero(api.Sub(asset, input.Asset))
        inSum = api.Add(inSum, api.Mul(match, input.Amount))
      }

      outSum := frontend.Variable(0)
      for _, output := range outputs {
        match := api.IsZero(api.Sub(asset, output.Asset))
        outSum = api.Add(outSum, api.Mul(match, output.Amount))
      }

      adjustedIn := inSum
      for i, publicAsset := range publicAssets {
        match := api.IsZero(api.Sub(asset, publicAsset))
        adjustedIn = api.Add(adjustedIn, api.Mul(match, publicAmounts[i]))
      }
      api.AssertIsEqual(adjustedIn, outSum)
    }
  }
```

Identical preimages also produce the same nullifier because nullifier derivation includes the commitment, blinding, and nullifier secret but not the leaf index. Spending either occurrence consumes the shared spend right and prevents a double spend. The resulting impact is a persistent availability failure in the official wallet and Photon proof-lookup path, where one duplicate may make the hash-only request fail without continued attacker activity.

## Remediation

Represent an output occurrence as `(tree, leaf_index, commitment)` throughout wallet state, client proof requests, and Photon. Photon should retrieve the requested tree position, verify that the stored leaf equals the supplied commitment, and return the proof for that position even when the same commitment exists elsewhere.

## Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

# 5. Suggestions

We identified 14 suggestions during our audit. While suggestions do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

| ID | Severity | Status | Description |
|---|---|---|---|
| OS-ZZK-SUG-00 | Informational | ○ Ack'd | Impossible Poseidon injectivity axiom invalidates Lean hash-binding proofs |
| OS-ZZK-SUG-01 | Informational | ✓ Resolved | Merkle-root gadget trusts inconsistent height and slice lengths |
| OS-ZZK-SUG-02 | Informational | ✓ Resolved | Signed-amount range check accepts one unreachable boundary value |
| OS-ZZK-SUG-03 | Informational | ✓ Resolved | Merkle-tree loader returns mutable bytes after releasing the account borrow |
| OS-ZZK-SUG-04 | Informational | ✓ Resolved | Generic tree initialization can record capacities that do not match its memory layout |
| OS-ZZK-SUG-05 | Informational | ○ Ack'd | Ordinary non-sentinel existing-value lookup already returns the documented predecessor |
| OS-ZZK-SUG-06 | Informational | ✓ Resolved | A malicious prover cannot replace the client-bound dummy-output owner marker |
| OS-ZZK-SUG-07 | Informational | ✓ Resolved | Noncanonical output commitments panic instead of returning a program error |
| OS-ZZK-SUG-08 | Informational | × To-do | Non-inclusion verifier discards a false Merkle verification result |
| OS-ZZK-SUG-09 | Informational | ○ Ack'd | Proof helper accepts ciphertexts too large for the transaction wire format |
| OS-ZZK-SUG-10 | Informational | ○ Ack'd | Proof bundle omits the instruction discriminator and circuit identity |
| OS-ZZK-SUG-11 | Informational | ○ Ack'd | Proof helper emits decimal view tags as noncanonical hex strings |
| OS-ZZK-SUG-12 | Informational | ○ Ack'd | Public indexed-array API panics on an out-of-range predecessor index |
| OS-ZZK-SUG-13 | Informational | ○ Ack'd | Deposits fail when Token-2022 assesses a positive transfer fee |

© 2026 Otter Audits LLC. All Rights Reserved.

## Impossible Poseidon injectivity axiom invalidates Lean hash-binding proofs

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-00 | Informational |

### Description

`Circuit.lean` defines `F` as `ZMod Order`, where `Order` is the BN254 scalar-field order, assumed prime. $F$ is a finite field with $q = \mathrm{Order}$ elements.

prover/server/formal-verification/FormalVerification/Circuit.lean

```
8  def Order : ℕ :=
   0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001
9  variable [Fact (Nat.Prime Order)]
10 abbrev F := ZMod Order
11 abbrev Gates := GatesGnark12 Order
```

`poseidon₂` in `Poseidon.lean` maps a two-element vector over $F$ to the single field element assigned by the extracted Poseidon circuit. Its domain contains $q^2$ vectors while its codomain contains only $q$ elements. Because $q > 1$, the pigeonhole principle guarantees distinct inputs with equal outputs, so the function cannot be injective.

prover/server/formal-verification/FormalVerification/Poseidon.lean

```
36 def poseidon₂ : Hash F 2 := fun a => (Poseidon_2_uniqueAssignment a 0).val
37
38 @[simp]
39 lemma Poseidon_2_iff_uniqueAssignment {v : List.Vector F 2} {k : F -> Prop}:
40     ZolanaProver.Poseidon_2 v (0:F) k ↔ k (poseidon₂ v) := by
41   unfold poseidon₂
42   apply Iff.of_eq
43   rw [(Poseidon_2_uniqueAssignment _ _).equiv]
44   rfl
45
```

`Main.lean` still declares `poseidon₂_collisionResistant` as an axiom and registers it as a global `Fact`. At the pinned ProvenZk revision, `CollisionResistant` requires equality of hash outputs to imply equality of the corresponding input vectors. For `poseidon₂`, this expresses information-theoretic injectivity, a far stronger claim than the computational assumption that an efficient adversary cannot find a collision. Since no function from $F^2$ to $F$ has the property, the axiom makes the surrounding Lean context inconsistent, since once `False` is derivable, false elimination proves any proposition.

© 2026 Otter Audits LLC. All Rights Reserved.

prover/server/formal-verification/Main.lean

```
18 axiom poseidon₂_collisionResistant : CollisionResistant poseidon₂
19 instance : Fact (CollisionResistant poseidon₂) := ⟨poseidon₂_collisionResistant⟩
```

The global instance supplies this impossible assumption to the Merkle and hash-chain results that require `Fact (CollisionResistant poseidon₂)`. This is used to invert hash-chain and public-input-hash equalities and to support circuit membership and non-membership correctness results. The affected theorems do not establish unconditional Merkle soundness or hash-chain binding. Any exported theorem whose axiom closure includes `poseidon₂_collisionResistant` inherits the same assurance defect. This concerns the guarantees the proof artifact establishes but does not result in a computable Poseidon collision or a defect in the runtime Poseidon gadget.

The formal-verification workflow extracts `Circuit.lean`, rejects circuit drift, and runs `lake build`. These checks confirm that the committed circuit matches the extraction and that Lean accepts the project, but they do not reject trusted axioms or inspect the axiom closure of exported theorems. Lean accepts the affected results because the impossible property enters as a trusted declaration and never becomes a proof obligation.

## Remediation

Remove `poseidon₂_collisionResistant` and its global `Fact` instance. Replace exact hash-inversion results with collision-extraction statements: equality of hash-chain outputs should yield either equal input vectors or explicit distinct Poseidon inputs with equal outputs, and Merkle soundness results should yield the intended membership conclusion or collision evidence. State computational collision resistance outside the finite-field function logic as an explicit cryptographic assumption, or formalize it through a bounded adversary and probability experiment. Add a finite-cardinality lemma establishing that no injection from $F^2$ to $F$ exists.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Merkle-root gadget trusts inconsistent height and slice lengths

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-01 | Informational |

### Description

The required construction invariant is `Height == len(Index) == len(Path)`. Each element of `Index` selects the left/right ordering for one Merkle level, while the corresponding element of `Path` supplies that level's sibling hash. However, as shown in `MerkleRootGadget`, the gadget accepts these three parameters with no cross-check and performs `Height` iterations without checking either slice length. A short slice stops compilation when indexed, whereas an oversized slice leaves elements unused. A nonpositive height emits no parent-hash rounds and returns the supplied leaf hash unchanged.

prover/server/circuits/gadget/proof_gadgets.go

```
23  type MerkleRootGadget struct {
24    Hash   frontend.Variable
25    Index  []frontend.Variable
26    Path   []frontend.Variable
27    Height int
28  }
29
30  func (gadget MerkleRootGadget) DefineGadget(api frontend.API) interface{} {
31    currentHash := gadget.Hash
32    for i := 0; i < gadget.Height; i++ {
33      currentHash = abstractor.Call(api, ProveParentHash{
34        Bit:     gadget.Index[i],
35        Hash:    currentHash,
36        Sibling: gadget.Path[i],
37      })
38    }
39    return currentHash
40  }
```

The same unchecked shape controls both computations in `MerkleRootUpdateGadget`, as shown below. The gadget first derives and asserts the old root, then derives the replacement root using the same `PathIndex`, `MerkleProof`, and `Height`. A malformed construction affects both the old-leaf membership assertion and the new-root computation.

prover/server/circuits/gadget/proof_gadgets.go

```
42  type MerkleRootUpdateGadget struct {
43    OldRoot     frontend.Variable
44    OldLeaf     frontend.Variable
45    NewLeaf     frontend.Variable
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    PathIndex   []frontend.Variable
    MerkleProof []frontend.Variable
    Height      int
}

func (gadget MerkleRootUpdateGadget) DefineGadget(api frontend.API) interface{}
{
    oldRoot := abstractor.Call(api, MerkleRootGadget{
        Hash:   gadget.OldLeaf,
        Index:  gadget.PathIndex,
        Path:   gadget.MerkleProof,
        Height: gadget.Height,
    })
    api.AssertIsEqual(oldRoot, gadget.OldRoot)

    newRoot := abstractor.Call(api, MerkleRootGadget{
        Hash:   gadget.NewLeaf,
        Index:  gadget.PathIndex,
        Path:   gadget.MerkleProof,
        Height: gadget.Height,
    })
    return newRoot
}
```

`Height` and the slice lengths are host-side Go values that configure the constraint system and witness schema during circuit compilation. They are not proof-witness values that an untrusted prover can vary after setup. Existing transaction and merge call sites use protocol heights of 32 and 40, and some proving-side input paths check the corresponding proof lengths. However, those checks do not enforce the invariant at the reusable gadget construction layer. The resulting impact is limited to trusted circuit construction as a malformed template may produce a shallower relation only if a setup or release operator generated matching keys and deployed its verifying key as though it represented the intended 32-level or 40-level tree.

## Remediation

Introduce one checked host-side construction path shared by `MerkleRootGadget` and `MerkleRootUpdateGadget`. Before emitting constraints, reject negative heights and require `len(Index) == len(Path) == Height`. Enclosing Zolana circuits should also require the applicable protocol height, 32 or 40, and permit zero only for a caller that defines a zero-level tree on purpose. Retain the existing proving-side proof-length checks, but do not rely on them to protect circuit setup.

## Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Signed-amount range check accepts one unreachable boundary value

| ID | Severity |
|---|---|
| OS-ZZK-SUG-02 | Informational |

### Description

Each public movement in an SPP transaction proof represents settlement legs aggregated by asset. As specified for `public_amounts`, the magnitude of every resulting signed amount must fit within `u64`, making the intended interval $[-(2^{64}-1), 2^{64}-1]$. The shared `RangeCheckSigned64` gadget shifts the field-encoded amount by $2^{64}$ and constrains the result to 65 bits. This rejects values above $2^{64}-1$ and below $-2^{64}$, but it permits $-2^{64}$: adding the offset to its BN254 field encoding gives zero, so an all-zero bit decomposition satisfies the constraint.

prover/server/circuits/spp_transaction/shared/balance.go

```
96  type RangeCheckSigned64 struct {
97    Value frontend.Variable
98  }
99
100 func (gadget RangeCheckSigned64) DefineGadget(api frontend.API) interface{} {
101   api.ToBinary(api.Add(gadget.Value, signedAmountOffset()), signedAmountBits)
102   return []frontend.Variable{}
103 }
104
105 func rangeCheckSigned64(api frontend.API, value frontend.Variable) {
106   abstractor.CallVoid(api, RangeCheckSigned64{Value: value})
107 }
108
109 func signedAmountOffset() *big.Int {
110   return new(big.Int).Lsh(big.NewInt(1), AmountBits)
111 }
```

The checked-in `TestSignedAmountRangeBoundary` preserves this asymmetric boundary by expecting $-2^{64}$ and $2^{64}-1$ to succeed while the next lower and higher values fail. Because `assertBalanceConservation` applies this gadget to every public amount, the circuit domain contains one value whose magnitude exceeds the documented `u64` limit.

This discrepancy does not enlarge the set of transactions the reviewed on-chain flow accepts. The program reconstructs signed per-asset aggregates from the settlement transfers, rejects an aggregate whose absolute value cannot convert to `u64`, encodes the bounded amount as a BN254 field element, and incorporates it into the proof input hash. The program can neither reconstruct nor bind a public amount of $-2^{64}$. The only issues may be specification drift and a shared range-check gadget that becomes unsafe if reused without equivalent host-side bounds and input binding.

© 2026 Otter Audits LLC. All Rights Reserved.

### Remediation

No security change is required for the reviewed flow. For exact circuit and host-domain parity, retain the 65-bit decomposition but also constrain $a + 2^{64}$ to be nonzero, excluding only $-2^{64}$. Update `TestSignedAmountRangeBoundary` so that $-2^{64}$ and $2^{64}$ fail while $-2^{64} + 1$ and $2^{64} - 1$ pass.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Merkle-tree loader returns mutable bytes after releasing the account borrow

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-03 | Informational |

### Description

An account-backed `BatchedMerkleTreeAccount` contains an exclusive mutable reference into Solana account data, so the account's dynamic mutable-borrow guard must remain live for the lifetime of that reference. The account type stores only a bare mutable `TreeAccountLayout` reference and has no field that can hold the guard `AccountView::try_borrow_mut` returns.

program-libs/batched-merkle-tree/src/merkle_tree.rs

```
47 pub struct BatchedMerkleTreeAccount<
48     'a,
49     const RH: usize,
50     const NUM_ITERS: usize,
51     const BLOOM: usize,
52     const ZKP: usize,
53 > {
54     pubkey: Pubkey,
55     pub(crate) layout: &'a mut TreeAccountLayout<RH, NUM_ITERS, BLOOM, ZKP>,
56 }
```

The safe public `address_from_account_info` method returns a `BatchedMerkleTreeAccount` parameterized by the implementation lifetime `'a`, but it does not borrow its `account_info` argument for that lifetime. Because the signature does not relate `'a` to the account argument, a caller may select a lifetime unrelated to the actual account borrow, including `'static`.

program-libs/batched-merkle-tree/src/merkle_tree.rs

```
1  impl<'a, const RH: usize, const NUM_ITERS: usize, const BLOOM: usize, const ZKP:
   usize>
2      BatchedMerkleTreeAccount<'a, RH, NUM_ITERS, BLOOM, ZKP>
3  {
4      /// Deserialize a batched address Merkle tree from account info.
5      /// Should be used in solana programs.
6      /// Checks that:
7      /// 1. the account owner is `program_id`,
8      /// 2. discriminator,
9      /// 3. tree type is batched address tree type.
10     pub fn address_from_account_info(
11         program_id: &[u8; 32],
12         account_info: &mut AccountView,
13     ) -> Result<BatchedMerkleTreeAccount<'a, RH, NUM_ITERS, BLOOM, ZKP>,
       BatchedMerkleTreeError>
14     {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
            Self::from_account_info::<ADDRESS_MERKLE_TREE_TYPE_V2>(program_id,
            account_info)
        }
        [...]
    }
```

The private account loader obtains a local mutable-borrow guard, constructs an `'a` mutable byte slice from its pointer, and derives the stored layout reference from that slice. The returned tree does not keep the guard, so the guard drops when the loader returns even though the tree still holds a mutable reference into the account data. The account API may then grant another mutable borrow of the same bytes while the first reference is live, so consumers of this safe API can hit undefined behavior without writing unsafe code themselves. Depending on later accesses and compiler behavior, this may surface as stale or lost writes, inconsistent tree or queue metadata, or an abort.

## Remediation

Remove the raw-slice lifetime extension from the account-backed constructor. Represent the stored layout so caller-owned byte slices retain an `&'a mut TreeAccountLayout`, while account-backed instances retain a `RefMut<'a, TreeAccountLayout>`.

## Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Generic tree initialization can record capacities that do not match its memory layout

ID
OS-ZZK-SUG-04

Severity
Informational

### Description

The generic batched Merkle-tree API permits `metadata.root_history_capacity` to differ from the const generic `RH`, even though nullifier safety requires these values to describe the same retention period. Both `init` and `init_from_layout` accept a runtime `root_history_capacity`, and `init_from_layout` stores it in metadata without checking it against the number of root slots the typed layout encodes.

program-libs/batched-merkle-tree/src/merkle_tree.rs

```
#[allow(clippy::too_many_arguments)]
pub fn init_from_layout(
    layout: &'a mut TreeAccountLayout<RH, NUM_ITERS, BLOOM, ZKP>,
    pubkey: &Pubkey,
    root_history_capacity: u32,
    input_queue_batch_size: u64,
    input_queue_zkp_batch_size: u64,
    height: u32,
    tree_type: TreeType,
    address_init_root: Option<[u8; 32]>,
) -> Result<BatchedMerkleTreeAccount<'a, RH, NUM_ITERS, BLOOM, ZKP>,
BatchedMerkleTreeError>
{
    layout.discriminator = Self::LIGHT_DISCRIMINATOR;
    let account_metadata = &mut layout.metadata;
    account_metadata.root_history_capacity = root_history_capacity;
    account_metadata.height = height;
    account_metadata.tree_type = tree_type as u64;
    account_metadata.capacity = 2u64.pow(height);
    account_metadata
        .queue_batches
        .init(input_queue_batch_size, input_queue_zkp_batch_size)?;
    [...]
}
```

`init_from_layout` initializes the physical root-history and hash-chain regions from the compile-time parameters `RH` and `ZKP`, as shown below. A layout with `RH` equal to 120 can retain 120 indexed roots while its metadata reports a capacity of 1, leaving inconsistent dimensions within the same account.

© 2026 Otter Audits LLC. All Rights Reserved.

program-libs/batched-merkle-tree/src/merkle_tree.rs

```
#[allow(clippy::too_many_arguments)]
pub fn init_from_layout([...]) -> Result<BatchedMerkleTreeAccount<'a, RH,
NUM_ITERS, BLOOM, ZKP>, BatchedMerkleTreeError>
    {
    [...]
    // Root history is a cyclic ring buffer. Upstream fills the entire ring
    // (length == capacity) on init, then seeds the first root. Write the
    // cyclic header `[current_index, length, capacity]`: capacity and length
    // are both ROOT_HISTORY; current_index advances to 1 when a root is seeded.
    layout.root_history.header[CYCLIC_LENGTH] = RH as u64;
    layout.root_history.header[CYCLIC_CAPACITY] = RH as u64;
    layout.root_history.header[CYCLIC_CURRENT_INDEX] = 0;
    if let Some(root) = init_root {
        if let Some(slot) = layout.root_history.data.get_mut(0) {
            *slot = root;
        }
        layout.root_history.header[CYCLIC_CURRENT_INDEX] = 1;
    }
    // Bounded hash-chain regions: length 0, capacity ZKP.
    for hash_chain in layout.hash_chains.iter_mut() {
        hash_chain.header[BOUNDED_LENGTH] = 0;
        hash_chain.header[BOUNDED_CAPACITY] = ZKP as u64;
    }
    [...]
}
```

This inconsistency affects Bloom-filter retirement. When the tree applies the final proof batch for queue batch B0 at sequence `s`, the update path passes the metadata capacity to `mark_as_inserted_in_merkle_tree`, which records B0's safety horizon as `s + metadata.root_history_capacity`. With a runtime capacity of 1, the recorded horizon is `s + 1` even though the physical root history still contains 120 slots.

After the first proof update for the next full batch B1 advances the live sequence to `s + 1`, Bloom cleanup may erase B0's filter once B0 is inserted and B1 is at least half full. Because B0's saved horizon then equals the live sequence, cleanup skips root invalidation. The generic API still exposes any physical root slot by index, so a downstream consumer that accepts caller-selected historical roots may keep roots that predate the nullifier while no pending Bloom filter still holds that nullifier.

Other mismatches may fail closed. With runtime capacity 2 and physical capacity 120, cleanup can reach its defensive root-cursor assertion with inconsistent indices, causing an on-chain update to abort and roll back on repeated attempts and preventing queue rotation. The duplicate-spend consequence also requires a value-bearing downstream integration that creates such malformed layouts, accepts caller-selected root slots, and relies on these Bloom filters for pending-nullifier protection; initialization alone does not bypass proof verification.

© 2026 Otter Audits LLC. All Rights Reserved.

The Shielded Pool does not expose this malformed state. Its `TreeAccount::init` requires the runtime root-history capacity to equal `NULLIFIER_RH`, requires the queue-to-proof-batch quotient to equal `NULLIFIER_ZKP`, and rejects noncanonical parameters before delegating to the generic initializer. However, because no production caller bypasses this wrapper, the issue is not severe despite the potential impact of misusing the public generic library.

## Remediation

Remove `root_history_capacity` as an independent generic-initializer argument and always store `RH` in metadata. Before writing any account bytes, reject zero queue dimensions, require `input_queue_zkp_batch_size` to divide `input_queue_batch_size`, and require their quotient to equal `ZKP`. Checked loading paths should also reject persisted metadata, root-history headers, hash-chain capacities, and queue dimensions that do not match the typed layout. Do not repair an affected value-bearing tree by changing its metadata in place, because cleanup may already have cleared its Bloom filters too early. Rebuild the tree from authoritative nullifier history, retire cached roots and proofs associated with the old account, and migrate consumers to the replacement.

## Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Ordinary non-sentinel existing-value lookup already returns the documented predecessor

ID
OS-ZZK-SUG-05

Severity
Informational

### Description

For an ordinary existing value `q`, the required low element is the node `p` satisfying `p.value < q` and `elements[p.next_index].value == q`. In a valid sorted list without duplicate inserted values, this makes `p` the greatest stored value below `q`, consistent with the protocol specification's definition of the low element in an indexed-tree adjacency witness. `IndexedArray` maintains two distinct orders. Elements occupy vector positions according to insertion order, while each `IndexedElement` stores a `next_index` that links elements in ascending value order. The vector position of the queried element need not identify its predecessor.

program-libs/indexed-array/src/array.rs

```
9  #[derive(Clone, Debug, Default)]
10 pub struct IndexedElement<I>
11 where
12     I: CheckedAdd + CheckedSub + Copy + Clone + PartialOrd + ToBytes +
       TryFrom<usize> + Unsigned,
13     I: Into<usize>,
14 {
15     pub index: I,
16     pub value: BigUint,
17     pub next_index: I,
18 }
```

`find_low_element_index_for_existent` follows the linked-list relationship by searching for a node whose successor has the queried value. When it finds that relationship, it returns the current node's vector position. For a valid array and a non-sentinel existing value, that node is the query's predecessor, not the queried node itself.

program-libs/indexed-array/src/array.rs

```
249 /// Returns the index of the low element for the given `value`, which is
250 /// already the part of the array.
251 ///
252 /// Low element is the greatest element which still has lower value than
253 /// the provided one.
254 ///
255 /// Low elements are used in non-membership proofs.
256 pub fn find_low_element_index_for_existent(
257     &self,
258     value: &BigUint,
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
) -> Result<I, IndexedArrayError> {
    for (i, node) in self.elements[..self.len() + 1].iter().enumerate() {
        if self.elements[node.next_index.into()].value == *value {
            let i = i
                .try_into()
                .map_err(|_| IndexedArrayError::IntegerOverflow)?;
            return Ok(i);
        }
    }
    Err(IndexedArrayError::ElementDoesNotExist)
}
```

`find_low_element_for_existent` passes the returned position to the array and clones the element stored there. The wrapper preserves the predecessor semantics that `find_low_element_index_for_existent` establishes. Changing it to return the queried element would violate the documented strict-lower-bound postcondition.

program-libs/indexed-array/src/array.rs

```
/// Returns the low element for the given `value`, which is already the
/// part of the array.
///
/// Low element is the greatest element which still has lower value than
/// the provided one.
///
/// Low elements are used in non-membership proofs.
pub fn find_low_element_for_existent(
    &self,
    value: &BigUint,
) -> Result<IndexedElement<I>, IndexedArrayError> {
    let low_element_index = self.find_low_element_index_for_existent(value)?;
    let low_element = self.elements[low_element_index.into()].clone();
    Ok(low_element)
}
```

The lower sentinel remains an underspecified boundary case. Index zero holds value zero, while a `next_index` of zero on the greatest node denotes the end of the list. Because the existent-value lookup dereferences that index and compares the resulting sentinel value with the query, querying zero on a nonempty array returns the greatest node even though zero has no strictly smaller predecessor.

program-libs/indexed-array/src/array.rs

```
pub fn find_low_element_index_for_existent(
    &self,
    value: &BigUint,
) -> Result<I, IndexedArrayError> {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    for (i, node) in self.elements[..self.len() + 1].iter().enumerate() {
        if self.elements[node.next_index.into()].value == *value {
            let i = i
                .try_into()
                .map_err(|_| IndexedArrayError::IntegerOverflow)?;
            return Ok(i);
        }
    }
    Err(IndexedArrayError::ElementDoesNotExist)
}

pub fn find_low_element_for_existent(
    &self,
    value: &BigUint,
) -> Result<IndexedElement<I>, IndexedArrayError> {
    let low_element_index = self.find_low_element_index_for_existent(value)?;
    let low_element = self.elements[low_element_index.into()].clone();
    Ok(low_element)
}
```

The semantic reversal does not occur for valid arrays and non-sentinel existing values as both helpers are read-only, and there is no production caller of either existent-value helper. The value-returning wrapper calls the index-returning helper, and the remaining in-tree calls are confined to a unit test.

## Remediation

Close the reported semantic-reversal issue without changing the ordinary lookup algorithm. Document that `find_low_element_index_for_existent` and `find_low_element_for_existent` require a non-sentinel member of a valid sorted list, and state the postcondition: the returned position `p` satisfies `elements[p].value < value` and `elements[elements[p].next_index].value == value`.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## A malicious prover cannot replace the client-bound dummy-output owner marker

ID
OS-ZZK-SUG-06

Severity
Informational

### Description

The `RingP256` circuit permits a dummy output's published owner marker to equal the hash of the shared P256 signing key's x-coordinate. As shown in `p256Authorization`, the circuit derives this hash from the canonical bit representation of the witnessed P256 x-coordinate. The surrounding `Define` flow adds the derived hash to the authorized participant set before validating masked dummy-output markers. Because the circuit enforces equality between a published marker and an output's actual owner only for real outputs, a dummy output may use this P256 participant marker while an unrelated nonzero marker remains invalid.

prover/server/circuits/spp_transaction/custom/p256.go

```
func (c *CustomRingP256Circuit) p256Authorization(
  api frontend.API,
) (frontend.Variable, frontend.Variable, frontend.Variable, error) {
  curve, err := sw_emulated.New[emulated.P256Fp, emulated.P256Fr](
    api,
    sw_emulated.GetCurveParams[emulated.P256Fp](),
  )
  if err != nil {
    return nil, nil, nil, err
  }
  point := sw_emulated.AffinePoint[emulated.P256Fp](c.Private.P256Pub)
  curve.AssertIsOnCurve(&point)

  fp, err := emulated.NewField[emulated.P256Fp](api)
  if err != nil {
    return nil, nil, nil, err
  }
  p256PkHash := hashBytes32Bits(api, fp.ToBitsCanonical(&point.X))
  [...]
}
```

This behavior does not expose the P256 identifier as a standalone Groth16 public signal. In `transaction`, `HashChain` folds the complete published-output marker vector into the preimage of `PublicInputHash`. Only `PublicInputHash` carries the public annotation in `CustomRingP256Public`. Assigning the P256 participant marker to a dummy slot does not place that marker in the proof response, and a remote prover cannot change an instruction-derived marker while keeping the original accepted public-input digest.

© 2026 Otter Audits LLC. All Rights Reserved.

prover/server/circuits/spp_transaction/custom/p256.go

```
func (c *CustomRingP256Circuit) transaction(
  api frontend.API,
  p256MessageHash frontend.Variable,
) shared.Transaction {
  return shared.Transaction{
    Shape:              c.Shape,
    Nullifiers:         c.Public.Nullifiers,
    OutputHashes:       c.Public.OutputHashes,
    UtxoTreeRoots:      c.Public.UtxoTreeRoots,
    NullifierTreeRoots: c.Public.NullifierTreeRoots,
    Inputs:             c.Private.Inputs,
    Outputs:            c.Private.Outputs,
    PrivateTxHash:      c.Public.PrivateTxHash,
    ExternalDataHash:   c.Public.ExternalDataHash,
    PublicAssets:       c.Public.PublicAssets,
    PublicAmounts:      c.Public.PublicAmounts,
    RingProgramID:      c.Public.RingProgramID,
    SignerPkHashChain:  gadget.RightHashChain(api, c.Public.SignerPkHashes),
    AllowDummyInputs:   c.Public.AllowDummyInputs,
    PublicInputHash:    c.Public.PublicInputHash,
    PreimageAfterPrivateTxHash: []frontend.Variable{
      p256MessageHash,
      c.Public.DefaultP256OwnerPkHash,
    },
    PreimageTail: []frontend.Variable{
      gadget.HashChain(api, c.Public.PublishedOutputOwnerPkHashes),
    },
  }
}
```

The standard transfer constructor assigns every dummy output the `confidential_view_tag()` of the first real input and emits the dummy tag inline. For a P256 owner, `confidential_view_tag()` returns the raw x-coordinate. A `RingP256` transfer whose first real input uses P256 and whose selected shape includes dummy outputs publishes that owner identifier without any prover manipulation. This does not establish signature forgery or unauthorized spending, but it may reveal the P256 owner identity behind the padding policy. The specified output relation allows a dummy output to publish either zero or a real participant identity to camouflage a confidential slot, as the private-output rules document.

## Remediation

Do not modify the circuit or rotate keys in response to the rejected malicious-prover disclosure theory alone. Document that the client fixes the statement before remote proving and that the remote prover already receives private witness data that circuit constraints cannot prevent it from disclosing out of band.

© 2026 Otter Audits LLC. All Rights Reserved.

### Patch

Resolved in fff18f4e5.

© 2026 Otter Audits LLC. All Rights Reserved.

## Noncanonical output commitments panic instead of returning a program error

ID
OS-ZZK-SUG-07

Severity
Informational

### Description

Every UTXO-tree leaf passed to Poseidon must be a canonical BN254 scalar satisfying $0 \leq \mathrm{int_be}(\mathrm{utxo_hash}) < r$, where

$$r = 21888242871839275222246405745257275088548364400416034343698204186575808495617.$$

The protocol specification requires output UTXO hashes to be well formed and equal to the circuit-computed commitments. A valid proof cannot bind a non-canonical output hash. The instruction wire format does not enforce this invariant on its own. `TransactOutput` represents `utxo_hash` as an unrestricted 32-byte array, and the owned and borrowed deserialization paths do not validate that its big-endian value is below the BN254 scalar modulus. Malformed bytes reach transaction processing even though proof verification would reject them later.

program-libs/interface/src/instruction/instruction_data/transact.rs

```
104 #[derive(Clone, Debug, PartialEq, Eq, SchemaRead, SchemaWrite)]
105 pub struct TransactOutput {
106     pub utxo_hash: [u8; 32],
107     pub owner_tag: OwnerTag,
108     #[wincode(with = "Option<containers::Vec<u8, FixIntLen<u16>>>")]
109     pub data: Option<Vec<u8>>,
110 }
```

The malformed value reaches the tree because `process_transact_ix` queues the input nullifiers and invokes `apply_output_tree` before calling `TransactProof::verify`. Within `append_batch`, Poseidon propagates each final leaf toward the root, but both hashing branches call `unwrap()` on the hasher result. For an instruction-supplied value such as `[0xff; 32]`, Poseidon rejects the non-canonical operand and `unwrap()` turns that expected error into a program panic instead of a stable shielded-pool error.

program-libs/tree/src/smt.rs

```
1 pub fn append_batch<'l, I>(&mut self, leaves: I) -> Result<(), TreeError>
2 where
3     I: IntoIterator<Item = &'l [u8; 32]>,
4 {
5     [...]
6     while let Some(leaf) = leaves.next() {
7         [...]
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
        for (subtree, zero_byte) in
        self.subtrees.iter_mut().zip(zero_bytes.iter()) {
            if current_index.is_multiple_of(2) {
                *subtree = current_level_hash;
                if !is_last {
                    break;
                }
                current_level_hash =
                    Poseidon::hashv(&[&current_level_hash, zero_byte]).unwrap();
            } else {
                let left = *subtree;
                current_level_hash = Poseidon::hashv(&[&left,
                &current_level_hash]).unwrap();
            }
            current_index /= 2;
        }
        [...]
    }
    Ok(())
}
```

Merge instructions have the same failure mode. `append()` delegates to `append_batch()`, while the shared `process_merge_core` flow applies the input-tree changes and appends the raw `output_utxo_hash` before invoking `MergeProof::verify`. This affects both default and ring merge flows that call the shared core. Proofless deposits offer no equivalent attacker-controlled path because the program computes their output commitments on-chain before the batch append.

A malformed submission aborts only its own Solana transaction. Transaction atomicity rolls back the preceding nullifier-queue and tree writes, so the malformed leaf, queued nullifiers, transfers, fees, and event do not persist. The issue is limited to malformed output bytes causing a generic program abort.

## Remediation

At every instruction boundary that accepts a caller-supplied output commitment, including transact variants and both merge paths, interpret each output hash as a big-endian integer and reject it unless it is strictly less than the BN254 scalar modulus before any nullifier insertion or UTXO-tree append. Also add a hasher-failure variant to `TreeError`, replace both Poseidon `unwrap()` calls in `append_batch` with explicit error propagation, and map that tree error to a stable `ShieldedPoolError` at the instruction boundary; using `?` alone is insufficient until the hasher error can be converted into `TreeError`.

## Patch

Resolved in fe36b7be0.

© 2026 Otter Audits LLC. All Rights Reserved.

## Non-inclusion verifier discards a false Merkle verification result

ID
OS-ZZK-SUG-08

Severity
Informational

### Description

A valid non-inclusion proof must establish that the queried value lies strictly between the indexed leaf's lower and upper bounds and that the trusted Merkle tree contains the reconstructed leaf. As shown in `verify_non_inclusion_proof`, the SDK checks both inequalities, reconstructs the indexed leaf hash, and invokes the ordinary Merkle verifier, but propagates only verifier errors and discards its Boolean result before returning `Ok(())`.

sdk-libs/merkle-tree/src/indexed.rs

```
pub fn verify_non_inclusion_proof(
    &self,
    proof: &NonInclusionProof,
) -> Result<(), IndexedReferenceMerkleTreeError> {
    let value_big_int = BigUint::from_bytes_be(&proof.value);
    let lower_end_value = BigUint::from_bytes_be(&proof.leaf_lower_range_value);
    if lower_end_value >= value_big_int {
        return
        Err(IndexedReferenceMerkleTreeError::NonInclusionProofFailedLowerBoundVio
    }
    let higher_end_value =
    BigUint::from_bytes_be(&proof.leaf_higher_range_value);
    if higher_end_value <= value_big_int {
        return Err(

            IndexedReferenceMerkleTreeError::NonInclusionProofFailedHigherBoundVi
        );
    }
    [...]
}
```

The inner `MerkleTree::verify` function returns an error for an out-of-capacity leaf index, an incorrect proof length, or a hashing failure. A well-shaped proof that reconstructs a root different from the receiver's current root returns `Ok(false)`. `verify_non_inclusion_proof` converts both a valid proof and a root-mismatched proof into the same successful result. `NonInclusionProof` also carries a `root` field, but the wrapper does not inspect it. Preserving and checking the inner Boolean would still bind verification to `self.merkle_tree.root()`.

If an external integration accepts an untrusted or stale `NonInclusionProof` and treats this method's success as authoritative, a path with valid bounds and shape can pass an absence check without any anchor to the current tree. A uniqueness, replay, or insertion decision that rests on that result alone could proceed from a false non-inclusion result. The current implementation does not expose that

© 2026 Otter Audits LLC. All Rights Reserved.

downstream impact. The only invocation of `verify_non_inclusion_proof` is a positive unit test. In `build_inputs`, Forester obtains non-inclusion witnesses from its reference tree before each sequential append and transfers their components into proof inputs, as shown below, but it does not call the faulty verification method.

forester/src/run.rs

```
fn build_inputs(
    reference: &mut ReferenceNullifierTree,
    next_index: u64,
    height: u32,
    leaves_hash_chain: [u8; 32],
    old_root: [u8; 32],
    batch_values: &[[u8; 32]],
) -> Result<(BatchAddressAppendInputs, [u8; 32])> {
    [...]
    for (offset, value_bytes) in batch_values.iter().enumerate() {
        let value = BigUint::from_bytes_be(value_bytes);
        let non_inclusion = reference
            .get_non_inclusion_proof(&value)
            .map_err(|err| anyhow!("non-inclusion proof: {err:?}"))?;
        low_element_values.push(BigUint::from_bytes_be(
            &non_inclusion.leaf_lower_range_value,
        ));
        low_element_indices.push(BigUint::from(non_inclusion.leaf_index as
        u64));
        low_element_next_values.push(BigUint::from_bytes_be(
            &non_inclusion.leaf_higher_range_value,
        ));
        low_element_proofs.push(path_to_biguint(non_inclusion.merkle_proof));
        new_element_values.push(value.clone());

        reference
            .append(&value)
            .map_err(|err| anyhow!("append nullifier: {err:?}"))?;
        [...]
    }
    [...]
}
```

No Shielded Pool instruction or Photon verification path reaches this method. The only effect is limited to incorrect behavior at the public SDK boundary.

## Remediation

Capture the Boolean returned by `self.merkle_tree.verify` and return a dedicated error, such as `NonInclusionProofMerkleRootMismatch`, when it is false. If `NonInclusionProof.root` remains

© 2026 Otter Audits LLC. All Rights Reserved.

part of the public API, require it to equal the receiver's trusted current root; otherwise, remove the field so callers cannot receive contradictory root metadata.

© 2026 Otter Audits LLC. All Rights Reserved.

## Proof helper accepts ciphertexts too large for the transaction wire format

ID
OS-ZZK-SUG-09

Severity
Informational

### Description

Each present output ciphertext must be representable by the protocol's `u16` length prefix, limiting its length to 65,535 bytes. As shown by `TransactOutput`, instruction serialization stores `data` in an optional byte vector whose length uses `FixIntLen<u16>`. A longer ciphertext falls outside the instruction field's domain, and the builder must reject it before it enters the proof statement.

program-libs/interface/src/instruction/instruction_data/transact.rs

```
100 /// One output slot in `transact` instruction data (spec: `transact`
101 /// `TransactOutput`): the output commitment, its owner tag, and an optional
102 /// ciphertext. `data: None` marks a slot covered by a preceding `Some` bundle
103 /// (a client/wallet placement convention); the program does not parse `data`.
104 #[derive(Clone, Debug, PartialEq, Eq, SchemaRead, SchemaWrite)]
105 pub struct TransactOutput {
106     pub utxo_hash: [u8; 32],
107     pub owner_tag: OwnerTag,
108     #[wincode(with = "Option<containers::Vec<u8, FixIntLen<u16>>>")]
109     pub data: Option<Vec<u8>>,
```

`ProofTransactionRequest` does not express or enforce this limit. Its `EncryptedUtxos` field is an unrestricted string, so the proof-bundle helper may receive a hex-encoded ciphertext whose decoded length exceeds the instruction format's maximum.

prover/server/prover-test/spp/prover/transaction/proof_bundle.go

```
1  type ProofTransactionRequest struct {
2    Name                  string                  `json:"name"`
3    InstructionDiscriminator uint8
     `json:"instruction_discriminator"`
4    ExpiryUnixTs          uint64                  `json:"expiry_unix_ts"`
5    SenderViewTag         string                  `json:"sender_view_tag"`
6    TxViewingPk           string                  `json:"tx_viewing_pk"`
7    Salt                  string                  `json:"salt"`
8    InterfaceTransfers    []InterfaceTransferRequest
     `json:"interface_transfers"`
9    EncryptedUtxos        string                  `json:"encrypted_utxos"`
10   StateEntries          []ProofStateEntry       `json:"state_entries"`
11   [...]
12 }
```

© 2026 Otter Audits LLC. All Rights Reserved.

`buildExternalData` decodes the complete `EncryptedUtxos` value with `parse.HexBytes` and proceeds without checking whether the resulting slice fits in a `u16`-length field. An oversized value advances into output resolution and external-data construction; nothing fails at the request boundary.

prover/server/prover-test/spp/prover/transaction/external_data.go

```
func buildExternalData(tx ProofTransactionRequest, outputHashes []*big.Int)
(externalValues, error) {
  [...]
  }
  encryptedUtxos, err := parse.HexBytes(tx.EncryptedUtxos)
  if err != nil {
    return externalValues{}, fmt.Errorf("encrypted_utxos: %w", err)
  }
  [...]
}
```

`resolveOutputs` assigns the decoded slice to the first real output and marks that output as containing data. It performs no length validation of its own, so an oversized ciphertext reaches callers that construct the proof's external-data hash.

prover/server/prover-test/spp/prover/transaction/external_data.go

```
func resolveOutputs(outputHashes []*big.Int, ownerTag [32]byte, encryptedUtxos
[]byte) ([]resolvedOutput, error) {
  outputs := make([]resolvedOutput, 0, len(outputHashes))
  for position, hash := range outputHashes {
    hashBytes, err := parse.FieldBytes(hash)
    if err != nil {
      return nil, fmt.Errorf("output hash %d: %w", position, err)
    }
    output := resolvedOutput{
      utxoHash: hashBytes,
      ownerTag: ownerTag,
    }
    if position == 0 {
      // The current request format carries one sender ciphertext bundle.
      // It is attached to the first output exactly like Some(data) in the
      // Rust TransactOutput; every following output carries None. hasData
      // is separate because Some(empty) must not collide with None.
      output.hasData = true
      output.data = encryptedUtxos
    }
    outputs = append(outputs, output)
  }
  return outputs, nil
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

`externalDataFieldHash` converts the output-data length to `uint16` but appends every byte of the original slice. For a 65,536-byte ciphertext, the encoded length wraps to zero while the complete ciphertext remains in the hashed preimage. The generated external-data hash commits to a malformed, out-of-domain representation that no instruction field can carry.

prover/server/prover-test/spp/prover/transaction/external_data.go

```
func externalDataFieldHash(data externalDataPreimage) *big.Int {
  [...]
  var outputSection []byte
  outputSection = binary.BigEndian.AppendUint16(outputSection,
  uint16(len(data.Outputs)))
  for _, output := range data.Outputs {
    outputSection = append(outputSection, output.utxoHash[:]...)
    outputSection = append(outputSection, output.ownerTag[:]...)
    if !output.hasData {
      outputSection = append(outputSection, 0)
      continue
    }
    outputSection = append(outputSection, 1)
    outputSection = binary.BigEndian.AppendUint16(outputSection,
    uint16(len(output.data)))
    outputSection = append(outputSection, output.data...)
  }
  [...]
}
```

The Rust `ExternalDataHash::hash` implementation has the same admission defect. It uses unchecked `as u16` conversions for output counts, output-data lengths, message counts, and message-data lengths while retaining all elements or bytes in the preimage. Both implementations construct the same malformed representation, so the two hashes agree with each other. The fixed-width instruction encoding is what cannot carry the ciphertext.

program-libs/interface/src/instruction/instruction_data/transact.rs

```
pub fn hash(&self) -> Result<[u8; 32], HasherError> {
    [...]
    preimage.extend_from_slice(&(self.outputs.len() as u16).to_be_bytes());
    for output in self.outputs {
        preimage.extend_from_slice(output.utxo_hash);
        preimage.extend_from_slice(&output.owner_tag);
        match output.data {
            None => preimage.push(0),
            Some(data) => {
                preimage.push(1);
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
                preimage.extend_from_slice(&(data.len() as u16).to_be_bytes());
                preimage.extend_from_slice(data);
            }
        }
    }
    preimage.extend_from_slice(&(self.messages.len() as u16).to_be_bytes());
    for message in self.messages {
        preimage.extend_from_slice(message.view_tag());
        preimage.extend_from_slice(&(message.data().len() as
        u16).to_be_bytes());
        preimage.extend_from_slice(message.data());
    }
    Sha256BE::hash(&preimage)
}
```

The proof-bundle path constructs the assignment from this external-data hash, calls `Prove`, verifies the resulting proof, and returns the original ciphertext in the bundle before a downstream consumer attempts instruction serialization. A 65,536-byte value costs at least 128 KiB of hex parsing, a 64 KiB decoded allocation, hashing, and a proof-generation and verification cycle, and the resulting artifact is unusable as an instruction. This confirms fail-late local resource consumption and a broken proof-export workflow. Sustained denial of service would also require the test helper to sit behind an untrusted request boundary and be invoked at the victim's expense. However, the current implementation has no such deployment.

## Remediation

Once `parse.HexBytes` decodes `EncryptedUtxos`, reject values whose decoded length exceeds `math.MaxUint16` before calling `resolveOutputs`, constructing the external-data hash, building the assignment, or invoking `Prove`. Change `externalDataFieldHash` to return an error and replace every narrowing conversion used for a `u16`-prefixed output count, output-data length, message count, or message-data length with a checked conversion so internal callers cannot bypass the entry-point check. Apply equivalent checked conversions in the shared Rust `ExternalDataHash::hash` implementation.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Proof bundle omits the instruction discriminator and circuit identity

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-10 | Informational |

### Description

A serialized proof bundle must identify the instruction route and verifying key required to consume its proof. `ProofTransactionRequest` accepts an `instruction_discriminator`, as shown below, but treats `name` as a separate caller-provided field.

prover/server/prover-test/spp/prover/transaction/proof_bundle.go

```
19 type ProofTransactionRequest struct {
20   Name                    string                    `json:"name"`
21   InstructionDiscriminator uint8
     `json:"instruction_discriminator"`
22   ExpiryUnixTs            uint64                    `json:"expiry_unix_ts"`
23   SenderViewTag           string                    `json:"sender_view_tag"`
24   TxViewingPk             string                    `json:"tx_viewing_pk"`
25   Salt                    string                    `json:"salt"`
26   InterfaceTransfers      []InterfaceTransferRequest
     `json:"interface_transfers"`
27   EncryptedUtxos          string                    `json:"encrypted_utxos"`
28   StateEntries            []ProofStateEntry         `json:"state_entries"`
29   Inputs                  []ProofInputRequest       `json:"inputs"`
30   Outputs                 []ProofUtxoRequest        `json:"outputs"`
31   UtxoTreeRootIndex       []uint16
     `json:"utxo_tree_root_index"`
32   NullifierTreeRootIndex  []uint16
     `json:"nullifier_tree_root_index"`
33   NullifierEntries        []string                  `json:"nullifier_entries"`
34   DataHash                string                    `json:"data_hash"`
35   RingDataHash            string                    `json:"ring_data_hash"`
36 }
```

During proof construction, `buildExternalData` copies the request discriminator into the external-data preimage, where it becomes the first component of `external_data_hash` and feeds the proof's public-input hash. `BuildProofBundle` emits only the proof-system shape, payer, and constructed transactions. Neither `ProofBundle` nor `ProofTransaction` preserves the discriminator or a circuit identifier. A consumer holding only the serialized bundle must recover this routing information from the original request or from external source-version context.

The bundle's input/output shape does not pin down the required circuit. `CircuitId` distinguishes `ConfidentialEddsa`, `RingEddsa`, `RingAuthority`, and `RingP256`, while encoding input, output, and public-asset-slot counts, as shown below. Multiple circuit families support overlapping shapes, and the validated selector determines both the proof layout and verifying key. This test helper compiles

© 2026 Otter Audits LLC. All Rights Reserved.

`CustomRingEddsaOnlyCircuit` today, but the emitted JSON neither declares nor versions that choice.

program-libs/interface/src/verifying_keys/circuit.rs

```rust
/// A supported `transact` circuit instantiation.
///
/// The tuple fields are `(number of inputs, number of outputs, number of
/// public asset slots)`. The selector is not a circuit public input: it selects
/// the verifying key and is validated against the dispatched instruction and
/// the instruction data before verification.
#[derive(Clone, Copy, Debug, PartialEq, Eq, SchemaRead, SchemaWrite)]
#[wincode(tag_encoding = "u16")]
pub enum CircuitId {
    /// NumInputs, NumOutputs, NumPublicAssets
    ConfidentialEddsa(u8, u8, u8),
    RingEddsa(u8, u8, u8),
    RingAuthority(u8, u8, u8),
    RingP256(u8, u8, u8, RingP256ProofData),
}
```

A bundle-only consumer may discard a valid proof, submit it through an incompatible instruction route, or pick the wrong circuit unless it recovers the request. Such mistakes may waste a proof-generation run and may incur a failed submission fee and retry work. They do not create a valid alternate transaction: the on-chain processor recomputes the external-data hash using the dispatched instruction discriminator, validates the circuit family and dimensions against that instruction, and selects the verifying key from the validated `CircuitId`. Because the affected helper resides under `prover-test` and its call sites are confined to test-package use, the issue is limited to a local artifact-contract defect with no accepted state-corruption or sustained denial-of-service path.

## Remediation

Version the proof-bundle schema and copy the validated instruction discriminator into every emitted `ProofTransaction`. Add a bundle-level circuit identity containing the circuit family, circuit version, complete shape including the public-asset-slot count, and a fingerprint of the verifying key. Populate these fields from the same `ProofSystem` metadata used to load or compile the proving key, not from `name`, and validate that the discriminator, circuit family, shape, and key identity agree before proving. Consumers should reject bundles with missing, unknown, or inconsistent identities.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Proof helper emits decimal view tags as noncanonical hex strings

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-11 | Informational |

### Description

A sender view tag has a single canonical bundle encoding (exactly 64 lowercase hexadecimal characters representing its 32-byte big-endian value). `BigInt` accepts unprefixed input as decimal and `0x`-prefixed input as hexadecimal. During proof construction, the builder parses `sender_view_tag` through this numeric path, checks it against the nullifier-tree domain, converts it to a zero-padded 32-byte value, assigns it to the resolved outputs as their owner tag, and folds it into `external_data_hash`.

prover/server/prover-test/spp/parse/fields.go

```
31 func BigInt(value string) (*big.Int, error) {
32   value = strings.TrimSpace(value)
33   if value == "" {
34     return nil, fmt.Errorf("empty field")
35   }
36   base := 10
37   digits := value
38   if len(value) >= 2 && value[0] == '0' && (value[1] == 'x' || value[1] == 'X')
     {
39     base = 16
40     digits = value[2:]
41     if digits == "" {
42       return nil, fmt.Errorf("empty hex field")
43     }
44   }
45   out, ok := new(big.Int).SetString(digits, base)
46   if !ok {
47     return nil, fmt.Errorf("invalid field %q", value)
48   }
49   return out, nil
50 }
```

The bundle response follows a separate encoding path. As shown in `buildProofTransaction`, the function constructs and verifies the proof before populating the returned `ProofTransaction`, but it fills the response's `SenderViewTag` from the original request string. The validated field value used during proof construction never reaches the response.

prover/server/prover-test/spp/prover/transaction/proof_bundle.go

```
1 func buildProofTransaction(ps *ProofSystem, tx ProofTransactionRequest,
  payerHash *big.Int) (ProofTransaction, error) {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
  if TransactionRequiresP256(tx) {
    return ProofTransaction{}, fmt.Errorf("spp: transaction %q uses the removed P256 ownership rail", tx.Name)
  }
  built, err := buildProofAssignment(ps.Shape, tx, payerHash, proofBuildOptions{})
  if err != nil {
    return ProofTransaction{}, err
  }
  assignment, publicInputs, publicInputHash, outputUtxos, transcript :=
    built.witness, built.publicInputs, built.publicInputHash, built.outputUtxos, built.transcript
  proof, err := Prove(ps, assignment)
  if err != nil {
    return ProofTransaction{}, err
  }
  if err := Verify(ps, assignment, proof); err != nil {
    return ProofTransaction{}, err
  }
  [...]
}
```

`HexString` trims whitespace and strips an optional hexadecimal prefix. It does not parse the input, validate its width, or zero-pad. An input of `9` comes out as the invalid one-nibble string `9`, an input of `10` is parsed for the proof as decimal ten but emitted as hexadecimal text meaning sixteen, and an input of `0x09` comes out as one byte instead of 32.

prover/server/prover-test/spp/parse/fields.go

```
func HexString(value string) string {
  value = strings.TrimSpace(value)
  if len(value) >= 2 && value[0] == '0' && (value[1] == 'x' || value[1] == 'X') {
    return value[2:]
  }
  return value
}

func FieldHex(value *big.Int) string {
  return fmt.Sprintf("%064x", value)
}
```

A strict consumer rejects these bundle tags as malformed. A permissive consumer that left-pads the emitted `10` obtains a value different from the one the proof committed to. Because the program recomputes `external_data_hash` from the transaction's resolved outputs before proof verification, the discrepancy fails closed and cannot alter the proved transaction.

© 2026 Otter Audits LLC. All Rights Reserved.

Although an affected request still incurs proof generation and local verification before producing an unusable artifact, no production caller or consumer exposes this local helper to untrusted requests at the pinned revision. As a result, the issue is limited to local producer-consumer failure.

## Remediation

Parse `sender_view_tag` once at the proof-bundle boundary and retain the validated field value or its `[32]byte` representation for both proof construction and response serialization. Serialize that retained value using `FieldHex` or `BytesHex` so the bundle always contains exactly 64 lowercase hexadecimal characters. Bundle consumers should reject any sender view tag that does not decode to exactly 32 bytes.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Public indexed-array API panics on an out-of-range predecessor index

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-12 | Informational |

### Description

The indexed, sorted array supplies the low and next values used to bracket each nullifier in a non-inclusion witness, as reflected by the proof-input layout in the specification. Its structure requires every dereferenced index to identify an element in `elements`, including supplied low indices, nonzero `next_index` values, `current_node_index`, and `highest_element_index`.

The public methods `new_element_with_low_element_index` and `append_with_low_element_index` do not enforce this invariant at their API boundary. Both index `elements` with the caller-supplied low index, and `append_with_low_element_index` also dereferences a nonzero successor while checking the ordering interval, as shown below. An absent entry causes a bounds panic. The existing `IndexedArrayError::IndexHigherThanMax` error never reaches the methods' `Result` return type.

program-libs/indexed-array/src/array.rs

```rust
pub fn append_with_low_element_index(
    &mut self,
    low_element_index: I,
    value: &BigUint,
) -> Result<IndexedElementBundle<I>, IndexedArrayError> {
    // TODO: add length check, and add field to with tree height here
    let old_low_element = &self.elements[low_element_index.into()];
    // Check that the `value` belongs to the range of `old_low_element`.
    if old_low_element.next_index == I::zero() {
        // In this case, the `old_low_element` is the greatest element.
        // The value of `new_element` needs to be greater than the value of
        // `old_low_element` (and therefore, be the greatest).
        if value <= &old_low_element.value {
            return Err(IndexedArrayError::LowElementGreaterOrEqualToNewElement);
        }
    } else {
        // The value of `new_element` needs to be greater than the value of
        // `old_low_element` (and therefore, be the greatest).
        if value <= &old_low_element.value {
            return Err(IndexedArrayError::LowElementGreaterOrEqualToNewElement);
        }
        // The value of `new_element` needs to be lower than the value of
        // next element pointed by `old_low_element`.
        if value >= &self.elements[old_low_element.next_index.into()].value {
            return
            Err(IndexedArrayError::NewElementGreaterOrEqualToNextElement);
        }
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    }
    [...]
}
```

`IndexedArray::default()` contains only the sentinel at index zero. Calling either raw-index method with `low_element_index` equal to one panics on its first array access, whatever the proposed value. In `append_with_low_element_index`, this access precedes any write to `highest_element_index`, `current_node_index`, or `elements`, so this malformed input leaves no partial indexed-array state.

If an external boundary passes untrusted indices to these methods, malformed input may turn a recoverable request failure into an unwind, worker failure, or process termination. The actul impact remains conditional since no non-test call site in the repository invokes these raw-index methods outside the indexed-array implementation.

## Remediation

Resolve the supplied low index and every nonzero successor with `get` or a shared checked-access helper, returning `IndexedArrayError::IndexHigherThanMax` whenever an entry is absent. In `append_with_low_element_index`, clone the resolved low element and validate its successor, the ordering interval, the prospective next node index, and relevant structural metadata before changing `highest_element_index`, `current_node_index`, or `elements`. Apply equivalent checked access to lookup slices and dereferences derived from `current_node_index` and `highest_element_index`, above all when accepting imported or otherwise untrusted array state.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

## Deposits fail when Token-2022 assesses a positive transfer fee

| ID | Severity |
| --- | --- |
| OS-ZZK-SUG-13 | Informational |

### Description

Zolana represents deposited assets as shielded UTXO value backed by SPL or Token-2022 collateral. As defined in the specification, a nominal deposit of $A$ requires the vault's transferable base balance to increase by $A$ so that the new shielded value remains backed. The interface-creation path breaks this requirement. `validate_token_mint_for_interface` permits mints carrying `TransferFeeConfig` and derives a vault layout containing the extensions required by the mint. As shown in `process_create_spl_interface`, registration remains subject to the configured permissionless-creation policy or protocol-authority check, but an authorized registration can still admit a transfer-fee mint.

programs/shielded-pool/src/instructions/create_spl_interface/processor.rs

```
pub fn process_create_spl_interface(accounts: &mut [AccountView], data: &[u8]) -
> ProgramResult {
    [...]
    let validated_mint = validate_token_mint_for_interface(mint,
    token_program.address())?;
    {
        let config = load_protocol_config(protocol_config)?;
        if !config.allows_permissionless_spl_interface_creation() {
            config
                .check_protocol_authority(authority.address())
                .map_err(ShieldedPoolError::from)?;
        }
    }
    [...]
}
```

During `settle_spl_deposit`, Zolana records the vault balance, invokes `TransferChecked` for the gross deposit amount $A$, and then requires the balance to equal its prior value plus $A$. When the active Token-2022 fee schedule produces a positive fee $F$, the transfer credits only $A - F$ to the vault's base balance and records $F$ as withheld fees. The equality check shown in `settle_spl_deposit` returns `PublicSettlementFailed` for every such deposit.

programs/shielded-pool/src/instructions/settlement/spl.rs

```
pub fn settle_spl_deposit(settlement: &SplDepositAccounts<'_>, amount: u64) ->
ProgramResult {
    [...]
    let interface_amount_after =
    token_account_amount(settlement.spl_interface_account)?;
    let expected_interface_amount = interface_amount_before
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
            .checked_add(amount)
            .ok_or(ShieldedPoolError::PublicSettlementFailed)?;
        if interface_amount_after != expected_interface_amount {
            return Err(ShieldedPoolError::PublicSettlementFailed.into());
        }
        Ok(())
    }
```

The balance measurement excludes the withheld fee by design. `token_account_amount` reads only the transferable base amount, as shown below. That is the right field for spendable collateral, and it is why the observed delta is $A - F$ while the committed deposit amount is $A$.

programs/shielded-pool/src/instructions/settlement/spl.rs

```
/// Read the transferable base balance. Token-2022 extension balances such as
/// `withheld_amount` deliberately do not count as vault collateral.
fn token_account_amount(account: &AccountView) -> Result<u64, ProgramError> {
    let data = account
        .try_borrow()
        .map_err(|_| ShieldedPoolError::PublicSettlementFailed)?;
    let state = PodStateWithExtensions::<PodAccount>::unpack(&data)
        .map_err(|_| ShieldedPoolError::PublicSettlementFailed)?;
    Ok(u64::from(state.base.amount))
}
```

The check fails closed, so the failed Solana instruction rolls back the token movement, withheld fee, shielded output, and tree update, and creates no undercollateralized value. Deposits for the registered mint stay unavailable whenever its effective fee is positive, including when a zero fee turns positive under a later fee schedule. Each submission is a one-shot failure. The user pays the transaction fee, but no token debit, withheld fee, shielded output, or tree update survives. Availability remains broken without continued request flooding because every later deposit with $F > 0$ follows the same path. The failure is confined to the affected mint and lasts while its effective fee remains positive.

## Remediation

Reject `TransferFeeConfig` in `validate_token_mint_for_interface` and migrate or disable any existing interfaces for mints carrying that extension. Existing affected interfaces should reject deposits before shielded-state construction with a fee-specific error. If transfer-fee mints must remain supported, determine the gross debit, transfer fee, net vault credit, and shielded amount before constructing proof public inputs, UTXO commitments, or events. Bind the same net collateral amount throughout those values and retain a post-CPI check that the transferable base-balance delta matches it.

## Patch

This issue was acknowledged by Zolana.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix A.

# Vulnerability rating scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

| Severity | Description |
|---|---|
| Critical | Vulnerabilities that immediately result in a loss of user funds or a full compromise of the protocol with minimal preconditions. |
| High | Vulnerabilities that could lead to a loss of funds or a compromise but require specific conditions to exploit. |
| Medium | Vulnerabilities that could lead to denial of service or incorrect accounting under realistic conditions. |
| Low | Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk. |
| Informational | Best practices to mitigate future security risks. These are classified as general findings. |

Resolution status is one of ✓ Resolved, ○ Ack'd, or × To-do.

© 2026 Otter Audits LLC. All Rights Reserved.

Appendix B.

# Procedure

We split every audit into two parts: design and implementation.

When auditing the design, we aim to ensure that the overall protocol itself is sound. Can someone forge a proof, break the privacy guarantees, or steal or double-spend funds by abusing the circuit logic or the cryptographic assumptions? To answer that, we need to know exactly what each circuit enforces and what the proving system and its hash and signature primitives guarantee. We also need to confirm that the public input hash binds every transaction parameter. For Zolana, this meant checking that the five circuit families agree with one another, and that no authorization path can be bypassed through ring-program forwarding or witness reuse.

A typical design bug would be a P256 signature gadget that leaves its hint values underconstrained, so a malicious prover can forge a signature without the private key. A flaw like that doesn't depend on the surrounding code or on the chain the protocol runs on. The constraint system just does not enforce what the protocol needs.

The implementation review covers how the circuits are compiled, how witness data is built, and how proofs are generated and then verified on Solana. Common problems here include witness construction errors, underconstrained gadgets, proof generation bugs, and verifying keys that have drifted from their proving keys. In Zolana, we also looked at how proofs are routed to the right shape across the five circuit variants, and whether root history stays consistent during batched nullifier and UTXO updates.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the circuit semantics, cryptographic guarantees, and the interaction between multiple proof paths.

We started by reading the specification, the circuit definitions, the formal verification theorems, and the on-chain verifier from end to end. Several auditors worked on the engagement together. Some focused on the overall architecture and others on low-level details, and we compared notes on constraint gaps, witness flow, verifier logic that diverges from the spec, and inconsistencies between circuit variants.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.